

RoR Research Log

Table of contents

2026-07-15	3
Proposal docx converted to Quarto	3
Fixed brms family-name bug: <code>student_t()</code> -> <code>student()</code>	3
2026-08-03	3
Outcome choice: model d_{ijk} , not final score conditional on consensus	3
Why not linear model on d_{ijk} ?	3
Simulator bug: scores weren't rounded to the CIHR-mandated tenth	4
Magnitude submodel switched from <code>student()</code> to <code>ordinal cumulative()</code>	5
Confirmed: reviewers can deviate from their own consensus score	5
This log created	6
2026-08-04	6
CIHR policy changes relevant to historical/trend data	6
CIHR Funding Analytics Team confirmed data availability and workflow	8
Simulations split by aim; Aim 2 simulation adds applicant characteristics	9
New document: <code>writing/ror-modeling-strategy.qmd</code>	10
Missing member-level heterogeneity (caught by Sam looking at a diagnostic plot)	10
Consensus was simulated directly instead of being derived from reviewers	12
2026-08-06	13
<code>score_min</code> was conflating two different things; streamlining moved to a selection filter ...	13
Decided not to simulate variable committee/application counts	14
Initial reviewer scores: too many very high values, tails asymmetric	14
Resolved: <code>re_formula = NULL</code> (on full data), not NA, for marginal effects	15
2026-08-11	16
New experimental streamlining DGP: <code>code/ror-sim-streamlining-experiment.R</code>	16
<code>ror-sim-aim1.R</code> vs. the streamlining experiment: the discussed-application score distributions look meaningfully different	17
2026-08-14	18
Checking committee size against real CIHR open data	18
CIHR's own funding-decision-process page confirms this and adds a real modeling gap . .	18
Application-level random effect was conflated across committees in all three simulation scripts	19
Pool-size draw simplified to a beta distribution, dropping the funded-count pipeline as a runtime dependency	20
Left tail of discussed applications' individual initial scores was too short	20
Recalibrated the exp (self-rated expertise) category mix	21
<code>m1_deviate</code> 's Truth column was wrong for all three random effects – they should be 0, not <u>u0c_sd/u0a_sd/u0m_bias_sd</u>	22

2026-08-19	22
ror-sim-deviate.R promoted to the canonical Aim 1 simulation	22
Discussed: should individual reviewers have a persistent	23
initial-scoring trait (some persistently harsh/lenient graders)?	23
Next step, not yet started: Appendix in ror-modeling-strategy.qmd walking through the full simulation script	24
writing/ror.bib citekey regression, again	24
Appendix added to writing/ror-modeling-strategy.qmd: full ror-sim-deviate.R script	25
2026-08-21	26
Expertise reference category switched from medium to high	26
Reviewer assignment now correlated with self-rated expertise	26
Committee member count (mem_n) now varies too, and is derived from pool_size	27
Model-specification equation added, and a three-format rendering saga	28
Magnitude model priors: tested an informed-threshold alternative, not adopting it	29
2026-08-25	30
Job-specific magnitude SD added to ror-sim-deviate.R: first treatment effect on the magnitude model	30
Aside: MASS::select() masking dplyr::select() broke the sim script mid-session	31
m1_magnitude couldn't see the job effect on magnitude: needed a disc (dispersion) submodel, not a location fixed effect	31
Cached the magnitude-model marginal predictions to output/, matching m1-deviate- me.rds's pattern	32
2026-08-26	33
ror-analysis-score-models.R split into a deviate script and a magnitude script	33
Flagged, not fixed: ror-analysis-magnitude-model.R has a duplicate model block and a tibble-recycling bug	33
Added the magnitude model's equation (html/typst) and a justification for ordinal over categorical	34
2026-08-27	35
Pinned down the exact mechanism of the marginaleffects hidden-attribute bug – earlier documentation of it was over-broad	35
Why the magnitude model's equation shows two disc coefficients for a binary covariate .	36
Updated the modeling-strategy doc's disc equation to 1 + job-style notation, for consistency with η	37
Actually fixed the typst image-path issue, instead of just working around it again	37

Chronological record of substantive decisions, rationale, and results for the “Influence of Reviewer Expertise and Engagement on Peer Review of Grants” project. This is the actual decision log for the project – edited by Sam, and by Claude when asked. For a lightweight session-continuity summary (where things stand, what to pick up next), see PROJECT.md at the repo root instead; that file is not a substitute for this one.

2026-07-15

Proposal docx converted to Quarto

Converted writing/cihr-ror-proposal.docx (a downloaded Google Doc believed to be the source of the funded proposal) into writing/cihr-ror-proposal.qmd, matching the repo's existing typst/Quarto template. All 38 numbered in-text citations were mapped to existing @keys in writing/ror.bib – no new bibliography entries were needed, every key already existed. The Gantt-style timeline figure was extracted from the docx and saved to writing/media/timeline.png. Rendered cleanly via `quarto render --to typst` with zero citation warnings.

Fixed brms family-name bug: `student_t()` -> `student()`

User hit Error in `student_t()` : could not find function "student_t" when sourcing code/ror-analysis-score-models.R. `student_t()` is a Stan distribution function name, not an R function brms exports – the correct brms family constructor is `student()`. Fixed the family = argument and the two comments referencing it.

2026-08-03

Outcome choice: model d_{ijk} , not final score conditional on consensus

Question raised: should the primary outcome be the consensus-to-final difference score (d_{ijk} , currently PROJECT.md's stated choice) or the final score modeled directly with consensus score as a covariate? Decided to keep d_{ijk} , for three reasons:

1. **The point mass at zero (many Rs taking consensus) doesn't disappear if you model final score instead – it just stops being a fixed reference.** A non-deviating member's final score equals consensus exactly. Modeling `final_score ~ consensus_score`
 - ...` still has that spike, just located at a value that moves application-by-application (wherever that application's consensus happened to land), instead of a single common value (0). Differencing first standardizes the spike location across every application, which is what makes a shared bernoulli-hurdle-style family tractable.
2. **Avoids reintroducing a confound.** A freely-estimated slope on consensus in a `final_score ~ consensus + ...` model has to recover something close to 1 from the data; if reviewer composition happens to correlate with consensus level across committees, that estimation can leak into the coefficients of actual interest. Differencing with an implicit slope of exactly 1 (which is the true administrative rule, not an assumption) removes that channel by construction.
3. **Matches the actual research question.** Aim 1 is about the *change* induced by discussion, not the *level* of the final score (which is mostly driven by application quality, not the process under study). Nothing is lost: `final_score = consensus + d_ijk` is a trivial identity, so anything Aim 3 needs at the score-level for funding-decision simulations can be reconstructed downstream.

Why not linear model on d_{ijk} ?

Empirical check on data/sim-deviation-data.csv (which has known ground truth – job/exp are coded to affect $P(\text{deviate})$ but not the *mean* deviation given deviation, since `dev_bias = 0` in `ror-sim-deviation.R`):

```

> m <- lm(deviation ~ job + exp, data = d)
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.0016      0.0016  -0.9956  0.3195
jobreviewer  -0.0014      0.0024  -0.5833  0.5597
explov       0.0040      0.0025   1.5784  0.1145
expmed       0.0017      0.0020   0.8430  0.3992
expnone      0.0052      0.0025   2.0649  0.0389

> m2 <- glm(deviated ~ job + exp, data = d, family = binomial())
              Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.9202      0.0337  -27.2977    0
jobreviewer  -0.2601      0.0484   -5.3764    0
explov       0.6631      0.0497   13.3402    0
expmed       0.4244      0.0410   10.3555    0
expnone      0.8639      0.0495   17.4691    0

```

The linear model's coefficients are tiny and mostly non-significant (largest effect size 0.005 points on a variable bounded at +/-0.5). The logistic model on the same covariates recovers the true, large, highly significant effect pattern baked into the simulator. This isn't a subtle power difference – because deviation magnitude is symmetric around zero given deviation, essentially all covariate signal lives in *whether* someone deviates, and a model that only looks at the conditional mean of d_{ijk} structurally cannot see it. Confirms the two-part (bernoulli x magnitude) model structure is necessary here, not over-engineering.

Simulator bug: scores weren't rounded to the CIHR-mandated tenth

Note that for getting the simulation up and running the CIHR reviewer scores (consensus and individual final scores) can only be entered to one decimal place. `ror-sim-deviation.R` was not enforcing this – consensus/score were raw floating-point draws (e.g. 3.808046), not multiples of 0.1.

Fixed by adding a `round_tenth()` helper and rounding consensus, deviation, and score to the nearest tenth. This introduced a correctness wrinkle: rounding a continuous deviation draw can collapse it to exactly 0, which is a contradiction for a row where `deviated == 1` (that member is defined to have deviated). Fixed with rejection sampling – now we redraw from `rtruncnorm()` and re-round, looping until no `deviated == 1` row has a zero deviation. With `dev_sd = 0.15`, about 26% of raw draws land in the collapse-to-zero band, so this loop does real work (average ~1.35 draws/row), not just handling a rare edge case:

```

> ptruncnorm(0.05, a=-0.5, b=0.5, mean=0, sd=0.15) - ptruncnorm(-0.05, a=-0.5,
b=0.5, mean=0, sd=0.15)
[1] 0.261

```

Added another check `stopifnot()` that checks confirming consensus/deviation/score are all exact tenths and that no `deviated == 1` row has zero deviation. Current distribution of deviation magnitude among deviators (n = 6756 deviators out of 18000 rows):

-0.5	-0.4	-0.3	-0.2	-0.1	0.1	0.2	0.3	0.4	0.5
8	69	333	1026	1915	1952	1033	340	75	5

Magnitude submodel switched from `student()` to `ordinal cumulative()`

Direct consequence of the rounding fix above: deviation magnitude among deviators is not continuous – it’s one of 10 discrete, unevenly-spaced values. Discussed the trade-off explicitly before changing anything:

- A continuous family (`student()/gaussian()`) assigns probability to impossible values, can’t represent the actual (very uneven) spacing between categories – sharply peaked near ± 0.1 , sparse at ± 0.5 – and has no natural bound at ± 0.5 without truncation hacks. The sparse tails are exactly the large swings most relevant to Aim 3’s funding-decision question, so a poorly-fit tail is a real cost, not a rounding curiosity.
- An ordinal `cumulative()` model matches the discretization exactly, gets the ± 0.5 bound for free, and (with `threshold = "flexible"`, the default) doesn’t assume the 10 steps are equally likely. Cost: contrasts in score-point units require an explicit probability-weighted sum over category values rather than falling out of a default linear prediction; 9 threshold parameters instead of 1 scale parameter (irrelevant at $n \sim 6700$).

Decided to implement it. Changes to `code/ror-analysis-score-models.R`:

- `deviation` recoded to an ordered factor over the 10 signed tenths (matched via `sprintf("%.1f", ...)` string comparison to avoid floating-point equality issues between the simulator’s rounding and a plain numeric `factor()` call).
- `m1_magnitude` now `cumulative(link = "logit", threshold = "flexible")` instead of `student()`.
- Priors switched to match the logit scale (`normal(0, 1.5)` on thresholds, `normal(0, 0.5)` on betas, matching `m1_deviate`’s own priors since both models are now logit-linked); dropped the `sigma` prior (no residual-SD parameter in an ordinal model).

Two follow-ups flagged in the script’s TODO block rather than resolved now: (1) the $E[\text{deviation}] = P(\text{deviate}) * E[\text{deviation} \mid \text{deviate}]$ combination step needs $E[\text{deviation} \mid \text{deviate}]$ computed as a probability-weighted sum over the 10 category values, not a linear prediction – needs an explicit `marginalEffects/tidybayes` contrast; (2) the 10-category $\pm 0.1 \dots \pm 0.5$ grid is still an assumption carried over from the simulator’s parameters, not yet confirmed with CIHR’s Funding Analytics Team.

Confirmed: reviewers can deviate from their own consensus score

Sam clarified the DGP: the 3 assigned reviewers must *agree* on a consensus score, but are not obligated to *use* it as their own final score – they can deviate from it just like non-reviewing panelists. Checked `ror-sim-deviation.R`: this was already correctly implemented. `p_dev = plogis(a0 + a1*panelist + ...)` only adds the `a1` term for panelists; reviewers still get a nonzero baseline `p_dev`. Confirmed empirically – reviewers deviate 32.7% of the time in the simulated data, not 0%. Consistent with the early proposal draft’s own language (`writing/proposal.qmd`:

“all committee members (including each of the reviewers) may assign their final score within a range of +/- 0.5 from the consensus score”). No code change needed.

This log created

Split the project’s actual decision/rationale record out from PROJECT.md. PROJECT.md will stay as a lightweight, Claude-oriented continuity note (where things stand, what to pick up next session) – it is not a substitute for a real research log and was never intended as one. This file is that log: chronological, decisions + rationale + evidence, editable by Sam directly and by Claude on request. Mirrors the format used in the u2-sibs project’s code/u2s-research-log.Rmd, adapted to Quarto.

2026-08-04

CIHR policy changes relevant to historical/trend data

Prompted by planning a meeting with CIHR’s Funding Analytics Team: if we incorporate historical/trend data from CIHR’s funding database, we need to know what changed in Project Grant policy over time so we don’t misread a policy-driven shift as a reviewer-behaviour effect (or vice versa). Searched CIHR’s public website and grey literature for dated policy changes. Full timeline, each with a direct link to CIHR’s own page:

- **2016 – Equalization introduced (early-career researchers).** CIHR began adjusting Project Grant outcomes so the proportion of early-career researcher NPIs funded is at least equal to their proportion of applications, when enough scientifically meritorious applications exist.
- **April 15, 2019 – Foundation Grant program sunset.** Message from the President: Final decision on the Foundation Grant program – after 4 competitions, CIHR ended the Foundation Grant program, citing “unintended consequences in funding distribution... not evident in the Project Grant program.” Foundation grantees have been gradually returning to the Project Grant applicant pool ever since – as of Fall 2025 this was still the 10th Project Grant competition Foundation holders (2014–2018 cohorts) could apply to. Real, multi-year compositional shift in who’s applying, not a one-time step change.
- **Spring 2021 – Budget expansion + equalization expanded (female NPIs, French-language applications).** ~\$50M/competition added as the reinvested Foundation budget flowed into Project Grant.
- **Fall 2022 – Committee restructuring.** Some peer review committees were dissolved due to low application pressure, with mandates absorbed into other committees. cmte identity/count is not stable across this point.
- **2022 – Indigenous Health Research (IHR) given a separate track.** Progress Update – Action Plan: Building a healthier future for First Nations, Inuit and Métis Peoples – distinct iterative peer-review process, developed with an external Reference Group (guiding principles from July 2022), reviews conducted through a mentorship lens. Per CIHR’s own equalization documentation, IHR applications are **excluded from the general ranking/equalization system entirely** (“the only exceptions being Indigenous health research, which has an iterative

review process and separate funding”). Any IHR committee data in a historical pull doesn’t belong in the same funding-decision model as everything else.

- **Fall 2024 – Equalization expanded (racialized, disability) + equalization zone redefined.** Removing barriers to funding for racialized people and persons with a disability – the substantial one. Racialized NPIs had received 82 fewer grants than application-share would predict over 5 competitions (21.9% of grants vs. 26% of applications); disability-identifying NPIs 17 fewer (3.2% vs. 4%). CIHR added both groups to equalization *and* redefined the equalization zone itself, from roughly ranks 15–34% to a narrower 10–25% window (top 10% always funded, bottom 75% never funded). This is a precisely dated, mechanical change to how “funded vs. not” is decided near the margin – directly relevant to Aim 3’s grey-zone question. CIHR references an internal evaluation of the intervention: Assessment of an Intervention to Equalize the Proportion of Funded Grant Applications for Underrepresented Groups at CIHR (Peer Review Congress abstract – worth tracking down the full paper/poster).
- **March 2026 – Active, undecided consultation on further changes.** Strengthening peer review together: Engaging the community to shape the future of the Project Grant program – dated 2026-03-16. CIHR convened ~90 committee Chairs/Scientific Officers (Feb 24–25, 2026) to discuss reintroducing in-person meetings (alternating with virtual), expanding Chair/Scientific Officer roles in reviewer assignment, formalizing reviewer participation as a “professional responsibility,” and moving toward a single annual competition with in-cycle early feedback. Application volume is up ~50% since Spring 2023; ~50% of invited reviewers decline. CIHR explicitly states no decisions have been made and more detailed proposals are coming “this spring”

(2026) – committee structure and/or virtual/in-person format could change again during our active grant period (Sept 2025 – Aug 2028).

Other pages consulted for context/background on this same search: CIHR commits to Project Grant changes based on research community recommendations, Peer Review Committee Mandates – Project Grant Program, Equity, Diversity and Inclusion in CIHR Funding Programs (Tri-Agency self-identification questionnaire launched May 2018; expanded 2020 for COVID-19 competitions, further expanded 2022 to 8 identity dimensions), Message from the Vice-President: CIHR’s commitment to enhancing equity, diversity, and inclusion.

Modeling implications flagged for the Funding Analytics Team meeting (see `writing/cihr-analytics-meeting.qmd`):

1. Historical/trend data spanning pre/post-2019 needs a Foundation-transition covariate or era indicator.
2. Committee identity should be treated as time-varying (Fall 2022 point).
3. IHR committee data likely needs to be excluded or modeled entirely separately.
4. Fall 2024 is a hard structural break for anything about the funding cutoff/grey zone – the definition of “grey zone” itself changed.
5. Given the March 2026 consultation is unresolved, worth asking directly whether committee mandates or review format are likely to change again within our data-collection window.

CIHR Funding Analytics Team confirmed data availability and workflow

Matthew Hogel (CIHR Funding Analytics) replied by email on 2025-12-04 to an earlier question about administrative data, working from the `sim-data` report we'd shared. Reviewed with Claude on 2026-08-04.

Confirmed 4-step workflow: (1) we send CIHR the cohort of interest (competitions/years) and a variable list; (2) CIHR sends back dummy data “aligned as much as possible to the true distribution” for the extractable variables in that cohort; (3) we send CIHR the R/python analysis code, built against the dummy data; (4) CIHR runs that code on the real data and returns results. This confirms the confidentiality workflow assumed in `PROJECT.md`, with one addition not previously nailed down: **CIHR will supply their own distribution-matched dummy data** for extractable fields, which is a real (if partial) alternative/complement to our own from-scratch simulation once step 1 happens.

Field-by-field availability, as extracts:

Available (extract)	Not available as extract, but usable via in-house CIHR-run analysis
Committee #, application #, member #, cid	Applicant gender
Job (identifies who did the at-home/primary review)	Applicant career stage
Experience (can be split into binary columns)	Applicant scientific productivity (“unlikely”)
Score (voted/final score)	Reviewer gender
Funding result	Reviewer “experience” (Matt asked us to clarify how we’d measure this)
Initial reviewer scores	Keywords/domain, # investigators, funding requested (all refused – identifiability risk)
Consensus score	
Resubmission status (2023 onward only)	
Applicant prior funding success (binary)	
Reviewer past funding success (binary)	
Reviewer conflicts of interest (probably – bundled with expertise data)	

The right-hand column isn’t a dead end: Matt explicitly committed a CIHR analyst to prep and run analysis in-house on those fields, using code and preprocessing instructions we supply – same remote-execution model, just without a dummy-data step first (still to be confirmed – see open questions below).

Implication for Aim 2: applicant gender and career stage – the two variables Aim 2 is built on – both fall in the “in-house only” column. We will very likely never see even distribution-

matched dummy versions of them, meaning the simulation is the *only* dry run we get before code runs once, unsupervised by us, against live confidential data. Raises the bar on getting `code/ror-analysis-score-models.R`'s handling of these covariates right before ever proposing a variable list to CIHR.

Decision needed, not yet made: whether/how to extend `code/ror-sim-deviation.R` with applicant-level `gender/career_stage` (application-level attributes) so Aim 2's model structure can be pressure-tested the same way Aim 1's already has been. Discussed with Sam 2026-08-04; proposal on the table (not yet implemented) is two categorical application-level covariates with both main effects and interactions with `job/exp` in both submodels, mirroring `u2-sibs`'s `treated:female` pattern – see chat for the open design questions (category counts, whether to bake in a true interaction effect for the simulation to recover).

Follow-up questions for Matt/CIHR, not yet sent:

1. For the fields in the “no extract” column – can CIHR's dummy-data step (workflow step 2) still include *synthetic* `gender/career-stage` columns (not real values) so we can pipeline-test against something, even though the real columns never leave CIHR? Or is the in-house-run path truly a black box until the real run?
2. “Resubmission status data only from 2023 onward” – does this mean our proposed cohort effectively has to start at the 2023 competition if we want that field, or can we request it from CIHR's raw records dated from 2023 forward for the years and let us handle it as a partially-missing covariate?
3. Clarify how we'd want “reviewer experience” measured, since Matt flagged that as the blocker to answering yes/no on it.

Simulations split by aim; Aim 2 simulation adds applicant characteristics

Decided to stop growing one shared simulation and instead give each aim its own, progressively more complex script: `ror-sim-deviation.R` stays as Aim 1's simulation, untouched; `ror-sim-aim2.R` is new and builds on it. Aim 3 is deliberately not started yet – it's a different kind of simulation (an intervention/counterfactual on top of the funding decision itself, e.g. partial randomization conditional on a funding threshold), not just an extension of the same generative process, and the design isn't settled.

`ror-sim-aim2.R` adds applicant-level `gender` (female/male) and `career_stage` (early/established) – one draw per application, constant across all of that application's member-rows, matching how consensus is already application-level. Motivated directly by the CIHR email above: since these two fields will never be extractable, not even as CIHR's own distribution-matched dummy data, this simulation is the only rehearsal the Aim 2 code gets before it runs once, unsupervised, on real data.

Per Sam's direction, baked in **real, sizeable interaction effects** for Aim 2 to detect (deliberately large – not estimates of real bias, purely to confirm the model machinery can recover an interaction if one exists): `job(panelist) x gender(female) = 0.5` and `exp(none) x career_stage(early) = 0.6`, both on the $P(\text{deviate})$ logit scale. Applicant main effects included too but kept modest (0.1, -0.1) since they aren't the effects of interest. Consistent with the Aim 1

finding above (signal lives in *whether* someone deviates, not magnitude), all new signal goes into the deviation-probability part only; the magnitude submodel stays null for applicant terms.

Added an empirical check at the bottom of the script (mirrors the Aim 1 lm/glm check): `glm(deviated ~ job*gender + exp*career_stage)` on the simulated data recovers both interactions cleanly at $n=18,000$ (`jobreviewer:gendermale = 0.569`, `expnone:career_stageestablished = -0.576` – sign flips vs. the parameters above are just R’s alphabetical reference-level choice, magnitudes match). Confirms the two-part model structure validated for Aim 1 also has the sensitivity to pick up an Aim 2 interaction, not just an Aim 1 main effect.

Not yet done: an Aim-2-specific version of `code/ror-analysis-score-models.R` (i.e. a `brm()` formula that actually includes the `job:gender/exp:career_stage` terms) – the two-part *simulation* is ready, the two-part *model script* for Aim 2 still needs to be written.

New document: writing/ror-modeling-strategy.qmd

Wrote a design/validation memo pulling together everything above into one document, per Sam’s request: describes the simulated data-generating process and the two-part model for Aim 1, the Aim 2 extension, the empirical recovery checks (embedded live – reads the existing simulated CSVs and re-runs the lm/glm checks, does not regenerate data or fit any `brm()` models), and open questions for both Arijit and CIHR. Explicitly framed as design + validation, not a full pre-analysis plan.

Dual output (format: `html` with `code-fold: true/code-tools: true/embed-resources: true`, and format: `typst` matching `cihr-ror-proposal.qmd`’s C059 template) so it can be read either as a single self-contained HTML file or a typeset PDF; both render cleanly (`quarto render --to html / --to typst`, zero warnings after fixing two duplicate chunk-label warnings). Intended audience order: Arijit first for feedback, then a revision to CIHR’s Funding Analytics Team, superseding the pre-Bayesian `sim-data.qmd` note sent to them in 2025.

Results section is a placeholder (“What comes next”) rather than actual posterior marginal effects, since no current `brms` fits exist – the two committed `.rds` files were already identified as stale (fit under the pre-ordinal `student()` spec) and untracked from git on 2026-08-04.

Missing member-level heterogeneity (caught by Sam looking at a diagnostic plot)

Sam plotted mean score $\pm$ SD by `memno` on the simulated data and noticed it was essentially flat – no member-level variation at all, beyond what role/expertise already account for. Real reviewers plausibly have a stable personal tendency (some habitually score more leniently, some more harshly) that’s independent of their assigned role or self-described expertise. Checked the DGP: correct, there was no such term anywhere. `p_dev` depends only on `panelist/exp_*`, and `dev_bias` (the mean of the deviation-magnitude draw) was a single constant (0) shared by every member – no source of a stable per-member identity across their multiple reviews.

Two things made this worth fixing rather than just noting: (1) both `brm()` formulas in `ror-analysis-score-models.R` already have `(1 | cid)` – the model has been “listening” for member heterogeneity the simulation never produced, so fitting it today would just show that variance

component shrinking to ~ 0 , which doesn't test anything (unlike the job/exp/gender/career_stage checks already validated); (2) the *original* writing/sim-data.qmd, before the consensus/deviation reframing, actually had this ($u0m_sd = 0.2$, a member-level random intercept on raw score) – it silently dropped out when the DGP was rebuilt around consensus + deviation.

Discussed two conceptually distinct traits this could be: **propensity** (some members break from consensus more often, regardless of direction – would enter p_dev) vs. **direction/bias** (some members, when they do deviate, consistently lean harsh or lenient – would enter dev_bias). Sam's description matched the second. Also confirmed explicitly: this should be *heterogeneity* (a distribution of member-level tendencies, population average still 0, consistent with the earlier symmetric-deviation finding), not a single fixed direction applied to everyone – that would just be $dev_bias \neq 0$, a different and much stronger claim.

Implementation, in both ror-sim-aim1.R and ror-sim-aim2.R (independent scripts, so the fix had to go in twice): added $u0m_bias_sd = 0.1$, and a per-member draw $u0m_bias \sim N(0, u0m_bias_sd)$, generated with `group_by(cid) |> mutate(u0m_bias = rnorm(1, 0, u0m_bias_sd))` – deliberately *not* `faux::add_ranef("member", ...)`, which was checked and confirmed to assign by the raw crossed “member” factor (1..24) and give the *identical* draw to every committee's “member 07”, silently conflating 50 different people who happen to share a slot number. `cid` (committee + member number) is the actual unique identity; grouping by it avoids that bug. $u0m_bias$ enters the deviation-magnitude draw's mean ($dev_bias + u0m_bias$) in both the main draw and the rejection-sampling redraw loop, then gets dropped before writing output, same as $u0c/u0a$.

Added an `lmer(deviation ~ 1 + (1 | cid))` check to both scripts (mirrors the existing `lm/glm` recovery checks) – recovers a `cid`-level SD of 0.111 (Aim 1) and 0.115 (Aim 2) against a true value of 0.1, with residual SD ~ 0.166 against $dev_sd = 0.15$ (the small inflation in both is expected, from the tenth-rounding discretization).

A plotting subtlety worth remembering: Sam's original diagnostic plot grouped by `memno` alone, which pools 50 different real people per x-axis category (`memno` is recycled 1-24 *within* every committee, `cid` is the true unique identifier). Checked empirically: even now that real member-level heterogeneity exists, the SD of `memno`-grouped mean scores is 0.006 (averaging 50 people together erases it), vs. 0.097 when grouped by `cid` instead. The original flat plot was real – there was no effect yet – but the same plot re-run today would *still* look flat regardless, for an unrelated reason (wrong grouping variable), which would be an easy false-negative trap when checking this kind of fix in the future.

Also fixed in passing: `ror-sim-aim1.R`'s `write_csv()` still pointed at the old filename `sim-deviation-data.csv` after the script itself was renamed from `ror-sim-deviation.R` – the header comment said `sim-data-aim1.csv` but the code didn't match. Now both agree.

Correction to the above, same day: Sam asked why `faux` couldn't be used at all for the member effect. Tested directly (`add_ranef("cid", u0m_bias = u0m_bias_sd)` on data that already has the `cid` column) – it works correctly, giving one independent draw per unique `cid`, same result as the manual `group_by(cid) |> mutate(rnorm(1, ...))` version (confirmed identical recovery

numbers on rerun). The actual problem was never “faux can’t do this” – it was specifically that `add_ranef("member", ...)` groups by the wrong column (the factor `add_random()` created, crossed with committee at only 24 levels total). `add_ranef()` handles an arbitrary existing column fine once pointed at the right one. Simplified both scripts to use `add_ranef("cid", ...)` instead of the manual `group_by/ungroup()`, matching the style already used for `u0c/u0a`.

Consensus was simulated directly instead of being derived from reviewers

Sam pointed out a real structural gap: consensus is supposed to come from the 3 assigned reviewers (they read, score, discuss, and agree a number), but the simulation had been generating consensus directly as $b_0 + u_{0c} + u_{0a}$ – a committee/application random effect with no connection to any actual reviewer score. Real consensus negotiation is more textured than a plain average (Sam’s example: R1 scores 3.9 and is persuasive but has “a hard stop at 4.0,” R2/R3 are both around 4.3 – consensus could land at 4.0, not the mean of 3.9/4.3/4.3). Agreed a plain mean is a reasonable simulation-level stand-in, with one hard requirement: consensus must never fall outside the range of what the 3 reviewers actually said.

Implementation, both `ror-sim-aim1.R` and `ror-sim-aim2.R`: relabeled `b0/u0c_sd/u0a_sd` as the application’s *true underlying quality* (not consensus itself) – this is what the 3 assigned reviewers each independently perceive, with their own pre-discussion noise (new parameter `init_sd = 0.3`, an illustrative placeholder like everything else, chosen roughly to match the spread in Sam’s example). `init_score` (initial reviewer score) is generated only for the 3 `job == "reviewer"` rows per application (NA for panelists, matching reality – panelists never submit an initial score); consensus is the rounded mean of those 3. Using `mean()` makes “never falls outside the 3 reviewers’ range” true by construction – no extra clamping code needed, since the mean of 3 bounded numbers can’t exceed their own bounds. Verified anyway with an explicit `stopifnot()` check across all 750 simulated applications per script, plus a check that exactly 3 non-missing `init_score` values exist per application. Required reordering the pipeline: `add_ranef("cmte", u0c)/add_ranef("application", u0a)` now happen *before* reviewer assignment (previously after), since `init_score` needs `u0c/u0a` to exist.

This also fills in a field CIHR confirmed is extractable (“Initial reviewer scores – yes,” per the 2025-12-04 email) that the simulation wasn’t producing before – `init_score` is kept in the output, not dropped like the latent `u0c/u0a/u0m_bias` terms are, since it’s a real observable field, not a DGP internal.

Reran both scripts’ existing recovery checks after the change (`job/exp` and `job:gender/exp:career_stage` interactions, member-level `cid` variance) – all still recover cleanly, confirming the consensus restructuring didn’t disturb anything downstream of it.

Not implemented, flagged as a natural future extension but out of scope for what was asked: a reviewer’s own `init_score` currently has no influence on their own later deviated/deviation draw (e.g. anchoring – a reviewer who scored well below the eventual consensus might be more likely to pull their final score back toward their own original view). Worth considering once the current, simpler version is validated further.

writing/ror-modeling-strategy.qmd is now doubly out of date (already flagged stale after the `u0m_bias` addition, now also missing the consensus-derivation change) – still needs a refresh before it goes to Arijit.

2026-08-06

score_min was conflating two different things; streamlining moved to a selection filter

Re-examined the `score_min = 3.5` clamp used on `init_score` and final score and concluded it was doing two unrelated jobs at once: (1) a true scale bound on an individual score, and (2) the streamlining/discussion- eligibility threshold. Those aren't the same thing – in practice an individual reviewer's initial score can land below 3.5 on an application that still gets discussed (e.g. [3.4, 4.1, 4.1], mean clears the bar even though one score didn't). Clamping every individual score at 3.5 was wrong on that basis alone, but it also had a concrete, checkable consequence: examining the simulated output directly showed an artificial pileup of values sitting at exactly 3.5 (a delta-function spike with zero variance among the piled-up rows – the signature of a clamp, not a real mode), rather than a smooth tail the way a genuinely bounded-but-not-clamped distribution should look.

Fix, in both `ror-sim-aim1.R` and `ror-sim-aim2.R`:

- Split the single `score_min/score_max` pair into `scale_min` (0, the scale's true floor) plus `score_min/score_max` (3.5/4.9, now used *only* as the eligibility range for the eligibility filter below – never as a per-score clamp). `score_max` legitimately does double duty as both the true ceiling and the eligibility ceiling, since 4.9 really is both.
- `init_score` is now clamped only at [`scale_min`, `score_max`] – individual reviewers are free to land below 3.5.
- Added an actual selection step: applications are generated in a candidate pool (`app_n_candidates = app_n * 2` per committee) with `u0c/u0a/reviewer init_score/consensus` as before, then filtered to those whose rounded consensus (mean of the 3 initial scores) falls in [`score_min`, `score_max`]; `slice_sample(n = app_n)` per committee keeps exactly the target count. A `stopifnot()` before the sampling step fails loudly and specifically if a committee ever has fewer than `app_n` eligible candidates, rather than letting `slice_sample()` error opaquely – 2x oversampling is comfortable headroom for the current parameters but is a real assumption that could need revisiting if `b0/variance` parameters change a lot later.
- Final score is now clamped at [`scale_min`, `score_max`] (0/4.9), not [`score_min`, `score_max`] (3.5/4.9) – the ± 0.5 rule around consensus is a real administrative constraint and can legitimately push a score below 3.5 or (near the ceiling) above 4.9, so only the true scale bounds should ever clamp it.
- `consensus` itself needs no clamp at all now – it's guaranteed to fall in [`score_min`, `score_max`] by construction of the selection filter, and the existing `stopifnot()` check confirming that range still holds as a check on the filter rather than a description of a clamp.

Verified the pileup is gone: re-checked the distribution of consensus values immediately around 3.5 in the regenerated data and confirmed 3.5 is no longer disproportionately common relative to its neighbors (e.g. 3.6) – consistent with a smooth, naturally-truncated distribution rather than an artificial spike.

Also bumped `init_sd` from 0.3 to 0.4 in both scripts – a modest increase to better reflect plausible reviewer-to-reviewer disagreement on the same application. `u0c_sd/u0a_sd` (between-application variance) left untouched this round.

Both scripts’ existing recovery checks (job/exp, job:gender/exp:career_stage interactions, member-level cid variance) all still pass after the change.

writing/ror-modeling-strategy.qmd is now triply out of date – still needs a refresh, now also for this change.

Decided not to simulate variable committee/application counts

Sam raised whether `app_n` (applications discussed per committee) and `mem_n` (members per committee) should vary across committees, rather than being fixed at 15 and 24 – both plausible in reality, and Bayesian partial pooling is well-suited to handling the resulting imbalance. Discussed without implementing; landed on leaving both fixed for now, for three reasons:

1. **Nothing in the analysis code depends on it.** `brms/lme4` handle unequal group sizes natively – the $(1 \mid \text{cmte})$, $(1 \mid \text{app})$, $(1 \mid \text{cid})$ random-effects specification doesn’t change based on whether the simulated data happens to be balanced. Unlike the tenths-discretization or two-part-model choices, there’s no code path that would behave differently or need to be caught by testing against imbalanced data.
2. **Real engineering cost.** The pipeline leans on `faux::add_random()`’s fully-crossed balanced-grid generation throughout; genuine imbalance would mean replacing that with per-committee manual generation (`purrr::map_dfr()`-style), and would compound with the streamlining filter added earlier today, which currently targets a fixed `app_n` per committee.
3. **No real basis for the imbalance parameters yet** (Sam’s own framing: “I don’t actually have much data on this, but I’m guessing...”), and committee #/application #/member # are all fields CIHR already confirmed as extractable – the true distribution will just be there in the real data once it arrives, without needing to guess at it now.

Flagged one legitimate future reason to revisit: a one-off illustrative exhibit (e.g. a shrinkage plot for Arijit, or a methods-section argument for why the hierarchical approach was chosen) would be a different motivation than pressure-testing the pipeline, and wouldn’t need to be baked into `ror-sim-aim1.R`/`ror-sim-aim2.R` themselves.

Initial reviewer scores: too many very high values, tails asymmetric

Even after the truncation fix above, `init_score` still had far too much mass near the ceiling – 150/2250 reviewers (6.7%) at exactly 4.9, ~220 (9.8%) above 4.7. Diagnosed two separate causes: (1) a real remaining clamp artifact – `init_score` was still `pmin(score_max, ...)`-clamped, which piles any raw draw above 4.9 into a spike exactly at the boundary, same mechanism as the `score_min` bug just fixed, just not yet applied here; (2) seems like a *symmetric* normal is probably the wrong

shape since in my experience reviewers are much more reluctant to award near-ceiling scores than they are to spread toward the floor.

Sam supplied an illustrative reference distribution (`ex_long$score`, approximates his experience, not real data, not meant to be matched exactly): skewness = -0.97, kurtosis = 5.16 (b2 convention, so excess ~2.16 – notably more peaked/heavier-tailed than normal), mean 4.10, median 4.1, SD ~0.42. Notable: mean and median are nearly identical despite the strong negative skew, which means the skew is driven by a *thin* tail of unusually low outlier scores rather than a broad leftward shift of the bulk of the distribution – a shape a simple two-piece normal can approximate the *direction* of but not the peakedness/kurtosis. Explicitly agreed not to chase the reference exactly (“no need to match directly... leave some room for uncertainty”).

Implementation, both `ror-sim-aim1.R`/`ror-sim-aim2.R`: kept the two-piece (split) normal for reviewer noise (SD below the true-quality center vs. SD above it), but **replaced the `pmin()/pmax()` clamp with proper rejection sampling** – `init_score` is generated unclamped, then any reviewer whose draw lands outside `[scale_min, score_max]` is redrawn (same while-loop idiom already used for the deviation-collapse fix), so out-of-range mass gets redistributed by redrawing rather than piling up at the edge. Grid-searched `init_sd_lo/init_sd_hi` against the illustrative target (accounting for `u0c_sd/u0a_sd` stacking on top of reviewer noise, which was inflating the tail more than reviewer noise alone would suggest) – landed on **`init_sd_lo = 0.40`, `init_sd_hi = 0.15`** (down from the symmetric `init_sd = 0.4/0.3` tried earlier same day). Resulting shape on the actual regenerated data: mean ~4.0-4.05, SD ~0.37-0.39 (slightly tighter than the pre-streamlining grid search predicted – the eligibility filter selectively excludes applications where reviewers scored low, which trims the observed lower tail too), median 4.1, $P(\text{score} > 4.7) \sim 1.5\text{-}3\%$, $P(\text{score} == 4.9) \sim 0.6\text{-}1.3\%$. Skewness of the resulting distribution (~-0.37, checked in a standalone grid-search script, not saved) is directionally correct but milder than the -0.97 reference – accepted per Sam’s explicit “don’t chase this exactly.”

Both existing recovery checks (Aim 2 interactions, member-level cid variance) still pass on the regenerated data in both scripts.

Resolved: `re_formula = NULL` (on full data), not NA, for marginal effects

The `re_formula = NULL` vs. NA choice flagged as an open TODO since the first draft of `code/ror-analysis-score-models.R` is now decided: **`re_formula = NULL`, evaluated on the full dataset**, is the convention for headline marginal-effects estimates (e.g. `avg_predictions()`) in this project, not `re_formula = NA` with a synthetic covariate grid. Two reasons:

1. Aim 1/2’s research question is about the general reviewer-expertise/ engagement relationship, not the specific 50 sampled committees – which sounds like an argument for NA (population-average), but `re_formula = NULL` averaged across the *full* observed data already *is* a population-average estimate – it marginalizes over random effects empirically, using the actual observed distribution of committees/members/applications, rather than analytically zeroing them out.
2. `re_formula = NA` is not a safe shortcut to “population-average” under a nonlinear (logit) link: zeroing random effects on the latent scale and then transforming gives the prediction for a

hypothetical median/typical cluster, not the *average* prediction across the cluster distribution – those differ under any nonlinear link (Jensen’s inequality), which is exactly our situation (bernoulli + ordinal-cumulative, both logit-linked).

Matches the precedent already set in `u2-sibs` (see that project’s `code/u2s-research-log.Rmd`, 2025-05-27 entry) – same fork was worked through there, landing on marginalizing over the observed random effects (`re_formula = NULL` on real data) rather than integrating them out via hypothetical new levels.

Orthogonal to this: `ndraws = <n>` (e.g. 200) in `marginalEffects` calls is a good way to speed up iteration during model-checking/simulation tweaking – confirmed working well in practice. That knob controls how many posterior draws get averaged over, not *what* is being estimated, so it’s fine to use a small `ndraws` for quick iteration and the full draw count for a number actually being reported. The `re_formula = NA + small-datagrid()` shortcut discussed alongside `ndraws` is fine for a fast approximate look during iteration, but the headline number should come from `re_formula = NULL` on the real data, per above.

2026-08-11

New experimental streamlining DGP: `code/ror-sim-streamlining-experiment.R`

Sam described the real CIHR streamlining procedure in more detail than was in the simulation: each of the 3 reviewers gives a separate categorical “top”/“bottom” call alongside their score (correlated with it, not determined by it), and an application is streamlined out iff ≥ 1 “bottom” call AND its mean-of-3 score ranks in the bottom 60% of its own committee’s candidate pool (relative rank, not a fixed absolute score). This directly explains a pattern found earlier the same day in a hypothetical Sam constructed: two applications with *identical* 3 reviewer scores had different discussed/not-discussed outcomes – a pure score-threshold rule can’t produce that, but this mechanism can (different top/bottom calls and/or different committee rank context at the same raw score).

Built as a new, explicitly EXPERIMENTAL script rather than folded into `ror-sim-aim1.R` – Sam was upfront about wavering between DGP fidelity and overcomplication, and the goal was to test whether the added realism changes anything material for recoverability, not to replace the validated pipeline. Structure: Stage 1 (new) simulates a genuine submission-volume candidate pool per committee (`pool_per_cmte = 40`, vs. `ror-sim-aim1.R`’s target-count-driven oversampling) and runs the top/bottom + rank streamlining decision; Stage 2 reuses `ror-sim-aim1.R`’s deviation-generating machinery unchanged, applied only to whatever survives Stage 1.

Also tried, then deliberately dropped: a “bring back” advocacy stage (a streamlined application reinstated by a persuasive reviewer, more likely when borderline on rank or when reviewers disagreed a lot). Sam’s call – felt too haphazard to model convincingly relative to what it bought. Checked before removing it: it only accounted for ~6% of discussed applications, and the same-score/different-outcome pattern it was meant to help explain already happens without it (confirmed by rerunning the “identical consensus, different outcome” check after removal – same

result, 6 distinct consensus values with both outcomes present, both before and after). It was also the least-grounded piece of the three-part mechanism – three invented coefficients vs. the top/bottom-vs-score relationship and the 60% threshold, both closer to stated fact than guesses.

Key finding: despite the much richer two-stage selection process determining which applications even reach discussion, Aim 1’s core recovery checks are unaffected – job/exp effects on $P(\text{deviate})$ and the member-level (cid) variance both still recover cleanly, at magnitudes essentially identical to `ror-sim-aim1.R`. Discussion rate ~40%, ~16 discussed applications per committee (SD ~2.2) – both in a plausible range given the two hypotheticals discussed earlier.

Read as: the added procedural realism doesn’t change the bottom line for Aim 1, so `ror-sim-aim1.R`’s simpler mechanism looks like a safe simplification for that purpose. Where the richer version would likely matter more is Aim 3 – who gets streamlined and who’s borderline is exactly the “grey zone” territory that aim is about.

ror-sim-aim1.R vs. the streamlining experiment: the discussed-application score distributions look meaningfully different

Sam plotted initial/consensus/overall score histograms side by side (`ror-sim-aim1.R` vs. `ror-sim-streamlining-experiment.R`) and the experiment’s distributions are visibly shifted right and missing the low tail across all three panels. Checked numerically rather than just by eye:

- `ror-sim-aim1.R` discussed applications: consensus range 3.5-4.8, mean 3.97.
- Experiment discussed applications: consensus range 3.7-4.8, mean 4.26. **87.5% of discussed applications are in the top 40% by rank** – the rank rule alone is doing almost all the selecting; only 12.5% got in from the bottom 60% by escaping a “bottom” call.

Mechanism, confirmed rather than assumed: `ror-sim-aim1.R` selects anyone clearing the fixed 3.5 floor, then takes a *random* subsample of 15 – a 3.5 and a 4.8 have equal odds of being chosen as long as both clear the bar, which preserves a broad, representative spread of the eligible pool. The experiment’s mechanism is structurally rank-selective (roughly top-40%), which concentrates the discussed sample toward the high end regardless of the absolute floor. Not a bug – a genuine, expected consequence of the mechanism being fundamentally different (competitive/relative selection vs. threshold-then-random-draw).

The more useful finding: checked which shape better matches the two hypothetical datasets from earlier the same session (the score/discussed-status examples Sam constructed to test the streamlining rule). In *both* hypotheticals, no “discussed” (Yes) application ever scored below ~4.0 (hypothetical 1’s lowest Yes was 4.03; hypothetical 2’s was 4.00). `ror-sim-aim1.R` routinely discusses applications scoring 3.5-3.9 – a third of its own range – which never shows up as a “Yes” case in either hypothetical. The experiment’s right-shifted, truncated shape lines up with that real-world anchor noticeably better than `ror-sim-aim1.R`’s broader spread does.

Not yet resolved: whether this means `ror-sim-aim1.R`’s simpler streamlining mechanism should itself be revisited (e.g. moving it toward something rank-based, or at least raising its effective floor) now that there’s evidence its discussed-application score range is too broad relative to the two hypothetical anchors – or whether to just leave `ror-sim-aim1.R` as the

intentionally-simple validated baseline and treat the experiment as the more-trustworthy-but-still- experimental alternative. Picking this up next session.

2026-08-14

Revising the simulation for the DGP and considering including varying the number of applications per committee, since we know this varies. Pulled down a couple of datasets from the Open Data (open.canada.ca) to see how the number of *funded* applications varies by committee, assuming something like a 15% funding per committee.

Checking committee size against real CIHR open data

Wrote `ror-cihr-committee-size-check.R` against a downloaded CIHR “Investments” file (2024-25, open.canada.ca, not committed to git – see `.gitignore`) to see whether real committee sizes support varying `pool_per_cmte/app_n` rather than holding them fixed.

Result: funded applications per committee per cycle (restricted to Project Grants and pooled across the 4 complete cycles, 229 committee-cycle observations): median 6, IQR 5-8, range 1-12, SD 2. Backing out implied total applications via funded/rate at 12%/15%/18%: medians 50/40/33, but wide spread even within one assumed rate (e.g. at 15%: IQR 33-53, range 7-80).

CIHR’s own funding-decision-process page confirms this and adds a real modeling gap

CIHR’s own description of the funding decision process (cihr-irsc.gc.ca/e/51315.html), which both independently validates the back-calculation above and surfaces something not currently in any of our simulations.

- CIHR states directly: **20-80 applications per committee**. Current back-calculated IQR (33-53 at 15%) sits inside that; the median (40) is almost exactly `pool_per_cmte`’s current fixed value in `ror-sim-streamlining-experiment.R`.
- “More than 2000 applications... across approximately 58 committees” implies ~34-40 average per committee.
- Confirms **3.5 as the fundable threshold** exactly, matching `score_min`
- One open discrepancy, not resolved: CIHR’s funding analytics database has **68** distinct Project Grant committees; CIHR’s text says “approximately 58.” Could be a rounding/older figure, or some of our 68 could be subcommittee splits (we saw e.g. “Behavioural Sciences - A/B/B-2/C” as four separate rows) that CIHR’s summary counts more coarsely. Not urgent, flagged for later.
- The “20-80” figure is total applications *reviewed* per committee – comparable to `pool_per_cmte` (candidate pool), not `app_n` (discussed count) – worth being precise about when calibrating.

Real modeling gap, not yet resolved: CIHR’s text states that “discussing an application can reconcile discrepant ratings and result in a final rating that is **lower than the average of the preliminary ratings**.” That’s an asymmetric claim about what discussion does – not “discussion adds symmetric noise around the pre-discussion average” (which is what `dev_bias = 0` currently encodes everywhere). If accurate, this is a real gap given the project’s core estimand is literally about what discussion does to scores. Not clear yet whether this describes the 3-reviewer

consensus step, the full-panel final average, or both – CIHR’s text doesn’t fully disambiguate, and that distinction matters for where a fix would actually belong. Discussed but explicitly deferred – next step is deciding how to pin this down before changing any `dev_bias` values.

Also clarified, not a new mechanism: “applications that are discussed are systematically ranked ahead of streamlined ones” is not a separate ranking rule – it falls directly out of “streamlined applications are not eligible for full grant funding” (categorical exclusion, regardless of score), which is exactly what `ror-sim-streamlining-experiment.R` already implements.

Decision: proceed with varying `pool_per_cmte` (total applications per committee), reversing the 2026-08-06 call to leave committee size fixed. That decision was explicitly conditional on “no real basis for the imbalance parameters yet” – we now have one, from two independent sources (CIHR’s stated 20-80 range and our own funded-count back-calculation) landing in close agreement.

Application-level random effect was conflated across committees in all three simulation scripts

While implementing the pool-size variation above, ran a minimal reproduction (`add_random(committee = ..., application = ..., member = ...)` followed by `add_ranef("application", u0a = ...)`) and found that `faux::add_random()`’s “application” factor is fully crossed with committee – it only has as many true levels as its own count (e.g. `app_n`), recycled identically across every committee, not one unique level per committee-application pair. Calling `add_ranef()` directly on that raw factor therefore gives *identical* `u0a` draws to same-labeled applications in different committees (e.g. `app == "1"` in committee 01 gets the exact same random draw as `app == "1"` in committee 02).

This is the same conflation bug class found and fixed for `member/cid` on 2026-08-05 – just never caught for `application` until now, in any of the three scripts that use this pattern. It went undetected this long because none of the existing recovery checks test `u0a`’s distributional correctness directly (they check downstream things like job/exp effects on deviation, or member-level variance, which don’t depend on `u0a` varying correctly by committee).

Fix, applied to all three scripts (`ror-sim-aim1.R`, `ror-sim-aim2.R`, `ror-sim-streamlining-experiment.R`): construct an explicit unique composite identifier (`aid = paste0(cmte, "_", app)`) and point `add_ranef()` at that instead of the raw crossed factor – the same fix pattern already used for `cid`. Verified directly in each script by checking that consensus (which depends on `u0a`) now differs across committees for the same `app` label, where before the fix it would have been suspiciously identical/correlated. All existing `stopifnot()` sanity checks and recovery checks (job/exp effects on `P(deviate)`, member-level `cid` variance) still pass in all three scripts after the fix.

For `ror-sim-streamlining-experiment.R` specifically, this fix was combined with the pool-size-variability implementation decided above, since both required rebuilding the committee/application/member grid from scratch anyway: `add_random(application = pool_per_cmte, ...)` (fixed pool per committee) was replaced with a manually-built grid (`reframe(app =`

`seq_len(pool_size), .by = c(cmte, pool_size)) + cross_join() with members), where pool_size is drawn per committee from output/cihr-funded-per-committee.rds via assumed_funding_rate = 0.15, bounded to CIHR's stated [pool_min_official = 20, pool_max_official = 80] range. Building the grid this way sidesteps the conflation bug by construction, since add_random() is no longer used for application at all.`

Result after both changes together: per-committee pool size drawn with median 40, IQR 33-52, range 20-73 (SD 12) – consistent with the back-calculation and CIHR's stated range above. Overall discussion rate 40%; discussed applications per committee median 16.5 (IQR 13-21, range 6-30). The same-consensus-different-outcome check still finds real examples (8 distinct consensus values with both discussed and not-discussed cases present).

Pool-size draw simplified to a beta distribution, dropping the funded-count pipeline as a runtime dependency

Immediately after the change above, Sam questioned why `assumed_funding_rate` belonged in this script at all, since the funding rate was only ever used to back out an implied *total submitted* pool size from real *funded*-application counts (in `ror-cihr-committee-size-check.R`) – and the data this project actually analyzes is *discussed*, not funded, applications, so the two shouldn't be conflated. On inspection the rate genuinely was only doing that one thing (sizing the upstream candidate pool; it had no bearing on the discussion or funding logic simulated later), but the dependency itself – reading `output/cihr-funded-per-committee.rds` and resampling from a bootstrap of only 229 real, fairly lumpy committee-cycle values – was more machinery than the “just a ballpark” purpose warranted.

Simplified to: `pool_size <- round(pool_min + (pool_max - pool_min) * rbeta(cmte_n, pool_shape1, pool_shape2))`, with `pool_min = 20`, `pool_max = 80` (CIHR's stated range) and `pool_shape1 = 2.5`, `pool_shape2 = 4` – shape parameters chosen (not fit) to roughly reproduce the mean/SD/right-skew the funded-count back-calculation had already shown (mean ~42, SD ~12): the beta draw gives mean 42.7, SD 10.1, median 41, IQR 34.5-49.75, range 25-71 in the latest run, close to the back-calculated distribution's mean 42.4, SD 11.7, median 40, IQR 33-52. `rbeta()`'s natural `[0,1]` support also means the scaled draw always lands in `[pool_min, pool_max]` with no `pmin/pmax` clamping needed.

This removes `ror-sim-streamlining-experiment.R`'s runtime dependency on `output/cihr-funded-per-committee.rds` and `assumed_funding_rate` entirely – the funded-count back-calculation in `ror-cihr-committee-size-check.R` remains useful as the evidence that informed the beta's shape, but is no longer something this script reads at run time. All checks (recovery, sanity, same-consensus/ different-outcome) still pass after the change.

Left tail of discussed applications' individual initial scores was too short

Plotting `init_score` for reviewer rows on *discussed* applications only (post-streamlining) showed a floor around 3.4, with nothing lower. Per Sam's own reviewing experience, a handful of discussed applications should carry at least one individual reviewer score down in the 2.0-3.0 range – one notably harsh/discordant read that the panel still moved forward on, with a still-defensible consensus.

Diagnosed as structural, not just a noise-SD tuning question. Because streamlining requires *both* ≥ 1 “bottom” call *and* the *consensus* ranking in the committee’s bottom 60% (section 3), a single reviewer’s very low score can only survive to discussion if the other 2 reviewers score high enough to pull the consensus back up near the committee’s median or better – an inherently rare combination regardless of how that one low score gets generated. Confirmed this by testing a small-probability “discordant outlier” mixture (some share of reviewers drawing from a much wider low-tail SD than everyone else): even at implausibly extreme settings (30% of all reviews, SD 1.3 vs. the baseline 0.30), only 1-2 out of ~2500 discussed reviewer-rows landed below 3.0 – the AND-rule’s own arithmetic, not the noise mechanism, was the binding constraint.

Fix: raised the shared `init_sd_lo` (the noise SD below an application’s true-quality center, applied to every reviewer, not a subset) from 0.30 to 1.1. This turned out to be the right lever specifically *because* it doesn’t need a whole reviewer to be a designated “outlier” – it just makes the ordinary low-tail noise wide enough that the occasional application clears the bar by chance (a high true-quality center absorbing one bad draw, or the committee’s own distribution being low enough that a moderate consensus still ranks well). `tb_slope` (tried lowering 6→4 as an intermediate step) turned out not to matter once `init_sd_lo` was doing the work, and was reverted to 6.

Calibrated the final value against a real reference distribution of individual-reviewer initial scores for discussed applications that Sam has access to (several hundred reviewer-scores across multiple past competition cycles) rather than guessing at a round number: that reference shows a median around 4.1, roughly 8% of individual scores below 3.5, and roughly 1.5% below 3.0, with meaningful within-application reviewer disagreement common (a nontrivial share of applications span a point or more across their 3 reviewers). At `init_sd_lo` = 1.1, the simulation’s discussed reviewer-level `init_score` lands close to that without exact fitting: median 4.2, 1st quartile 3.9 (matches), ~7.4% below 3.5, ~0.9% below 3.0, consensus floor at 3.6 vs. the reference’s 3.3 – reasonably close given “no need to chase this exactly” is the standing calibration philosophy throughout this project. The bulk of the distribution (median, IQR) is essentially unchanged from before this whole left-tail investigation started.

Recalibrated the exp (self-rated expertise) category mix

The `exp` sampling shares in `ror-sim-aim1.R`/`ror-sim-aim2.R`/`ror-sim-streamlining-experiment.R` had been an arbitrary illustrative split since the project’s earliest simulations (6 high / 10 medium / 4 low / 4 none out of 24 members – 25/42/17/17%), with medium treated as the dominant category and high fairly common.

Checked this against a real reference distribution of self-rated expertise Sam has access to (source kept out of this log per standing practice, as with other real-data calibration checks this session). The real shape is close to inverted relative to what the simulation assumed: “not enough” and “low” together make up the large majority of self-ratings, “high” is the rarest category by a wide margin – Sam’s read is that this reflects a real, fairly consistent pattern (reviewers are reluctant to claim medium/high expertise), not an artifact of one unusual meeting.

Changed to: 2 high / 5 medium / 7 low / 10 none out of 24 members (8.3/20.8/29.2/41.7%), approximating a target split of roughly 10/20/30/40. Applied identically to all three simulation scripts

for consistency, since this is a property of the reviewer population, not something that should vary by which experimental variant is running. Re-ran all three after the change – all sanity/recovery checks still pass, and the regenerated data/sim-data-aim1.csv matches the intended shares closely (8.2/29.8/20.5/41.5% observed vs. 8.3/29.2/20.8/41.7% target).

m1_deviate's Truth column was wrong for all three random effects – they should be 0, not u0c_sd/u0a_sd/u0m_bias_sd

After fixing the application/aid grouping bug in the simulation scripts, refit m1_deviate two ways to compare: (1 | app) (the old, conflated grouping) vs. (1 | aid) (the fixed, unique-per-committee grouping). Both gave essentially identical, small posterior SDs for every random effect (Application ~0.03-0.04, Committee Member ~0.05-0.06, Committee ~0.02), which looked at first like a recovery failure given the Truth column added earlier this session (which had those rows set to u0a_sd = 0.3, u0m_bias_sd = 0.1, u0c_sd = 0.1).

Ran a clean, minimal check first to rule out “bernoulli outcomes just can’t detect small variances at this sample size”: simulated a bernoulli GLMM with the exact same structure as the real fit (778 groups, 24 observations/group, true SD 0.3, baseline logit -0.8) and glmer() recovered it almost exactly (mean 0.292 across 15 replications) – so weak identifiability from binary data wasn’t the explanation.

The real explanation: p_dev (which generates deviated via rbinom()) is plogis(a0 + a1*panelist + a2*exp_high + a3*exp_low + a4*exp_none) only – no u0c, u0a, or u0m_bias term anywhere in it. There is no true committee-, application-, or member-level heterogeneity in *whether* a member deviates at all in the current DGP; u0c/u0a only drive init_score/consensus (an entirely different part of the DGP), and u0m_bias only enters the *magnitude* of deviation given deviation occurs (Part 2’s ordinal model, not this bernoulli one). So the true SD for cmte, cid, and aid in m1_deviate specifically is 0 for all three, which is exactly why app and aid looked identical: with no real signal for either grouping to find, a conflated grouping variable can’t matter.

Fixed: truth in code/ror-analysis-score-models.R now sets aid/cid/cmte to 0 for the m1_deviate table (was incorrectly reusing the score-level/magnitude-level SD parameters). Regenerated output/m1-deviate-table.rds from the existing cached aid-based fit (no refit needed, since the truth values don’t depend on the model fit) and confirmed the small estimates (0.038/0.053/0.019, all with CIs touching ~0) now read as correct recovery of a true value of 0, not underestimation. Added a note to this effect in writing/ror-modeling-strategy.qmd. Part 2 (m1_magnitude, once fit) is the one place cid should show a genuine non-zero truth (u0m_bias_sd = 0.1) – cmte/aid stay at 0 there too, since neither enters the deviation-magnitude formula either.

2026-08-19

ror-sim-deviate.R promoted to the canonical Aim 1 simulation

The experimental streamlining mechanism (top/bottom calls + rank-based selection, previously ror-sim-streamlining-experiment.R) is now the main Aim 1 DGP going forward, under code/ror-sim-deviate.R – not a side-by-side comparison against ror-sim-aim1-old.R’s simpler

fixed-threshold mechanism anymore. `code/ror-analysis-score-models.R` now reads `data/sim-deviate.csv` accordingly (was pointed at the older `data/sim-data-aim1.csv`, which had gone stale relative to the actual script in use – caught mid-session when the two didn’t match).

Also cleaned up: `ror-sim-deviate.R` no longer drops `aid` in its final `select()`. Unlike `u0c/u0a/u0m_bias/p_dev` (true latent DGP quantities that should never appear in an “observed” analytic dataset), `aid` is just `paste0(cmte, "_", app)` – a legitimate observable identifier a real dataset would have. It had been swept into the same drop line by pattern-matching on “used to build `add_ranef()`,” not because it belonged there. (Not yet done: the now- redundant `aid = paste0(cmte, "_", app)` reconstruction in `dl’s mutate()` in `ror-analysis-score-models.R` could be removed too, now that the CSV carries `aid` directly – harmless as-is since it recomputes the identical value, just duplicated logic.)

Caught mid-edit while wrapping up: the truth vector’s last three entries (`aid/cid/cmte`) had lost their `= 0` – Sam was mid-way through reworking this himself, prompted by not liking how plain `0.000` reads in the rendered Truth column, and hadn’t finished when the session ended. In the interim state, `c("Intercept" = -0.8, ..., "aid", "cid", "cmte")` silently coerces the whole vector to character (can’t mix named numeric and unnamed string elements), turning those three into unnamed values instead of names mapped to `0` – `truth["aid"]` etc. would silently return `NA` if the script ran like that. Restored `plain = 0` so the script runs correctly in the meantime, but **Sam still wants to revisit the display** (e.g. showing something other than a bare “0.000” for a structurally-zero random effect, maybe with a note explaining why) – not a value/correctness question, a presentation one. Pick this up before treating the Truth column as finished.

Discussed: should individual reviewers have a persistent

initial-scoring trait (some persistently harsh/lenient graders)?

Sam’s reviewing experience: yes, this is a real, recognizable pattern, distinct from the already-modeled `u0m_bias` (which only affects the *discussion-deviation* step, not initial scores at all – currently no persistent per-member trait feeds into `init_score` anywhere; the per-reviewer noise there is redrawn fresh for every review instance).

Decision: not worth building in for Aim 1/2. The project’s primary estimand is d_{ijk} (the discussion-deviation), and a persistent initial-scoring trait would mostly reshape *consensus* (since consensus is the mean of 3 reviewers’ scores) rather than the deviation mechanism itself – nothing about a reviewer’s own initial-scoring tendency currently needs to, or would naturally, feed into whether/how much they personally deviate during discussion, unless we specifically wanted to model a *different* hypothesis (e.g. “reviewers who scored unusually harshly relative to co-reviewers are more likely to hold their ground in discussion”) – a real, separate, and more specific claim, not something to bundle in under general realism.

Flagged for later: likely worth it for Aim 3. Funding-decision simulations depend on absolute score/consensus levels, where a persistently harsh reviewer could genuinely tip a real application across the streamlining or funding threshold – a case where the added realism could change a

substantive conclusion, not just add cosmetic texture. Aim 3's simulation hasn't been started yet; revisit this when it is.

Independent sanity check, in passing: Sam's stated real-world figures (~5-6 total application reviews per member per cycle, ~40% reaching discussion) line up well with two numbers calibrated separately this project: a beta-distributed candidate pool averaging ~42 applications/committee gives an expected $42 \times 3/24 \approx 5.25$ reviews/member, and the streamlining mechanism's discussion rate has been landing around 36-40% in recent runs. Two independently-derived figures converging on Sam's real-world number is a good sign, not acted on further.

Next step, not yet started: Appendix in `ror-modeling-strategy.qmd` walking through the full simulation script

The simulation has grown complex enough that Sam wants it documented in more depth than the current "Data structure" section. Discussed two options: a separate standalone `.qmd`, or an Appendix section at the end of `ror-modeling-strategy.qmd`.

Decided: Appendix in the existing document, not a separate file. Two reasons: (1) the document's own stated purpose ("lays out the data-generating process we believe matches CIHR's Project Grant peer review") already covers a full script walkthrough – it's in scope, not a tangent; (2) a second standalone document is one more artifact that can silently drift out of sync with the actual script, which is exactly the failure mode hit twice already today (`sim-data-aim1.csv` vs. `sim-deviate.csv`; `ror-sim-aim1.R` vs. `-old.R` vs. `-deviate.R`).

Planned mechanism: pull the actual script content into the appendix via Quarto's `file = chunk` option (e.g. `{r, file = here("code", "ror-sim-deviate.R")}`), rather than manually retyping or copy-pasting code into the `qmd` – so the appendix always reflects whatever the script currently says, with explanatory prose written around/between sections rather than as a frozen snapshot.

Not started yet – this is the next thing to pick up.

writing/`ror.bib` citekey regression, again

Pulling the latest office commits (8be0df2) reintroduced the same Better BibTeX citekey-corruption pattern fixed earlier this week: a fresh Zotero export renumbers citekeys based on collisions across Sam's *entire* library, not just this file, so ~42 entries picked up spurious trailing letters (e.g. `nandi2022a` where the in-file key was `nandi2022`) even though nothing about those entries changed. Diagnosed and fixed the same way as before: confirmed only one genuine in-file collision (`governmentofcanada2016`, two distinct entries), then stripped the spurious trailing letter from the other 42 via a protect-then-strip script, leaving the collision pair (`governmentofcanada2016/governmentofcanada2016a`) untouched. As before, this fix is local to the file and will likely be undone by the next Zotero export – the durable fix is on the Zotero side (pin citekeys so BBT stops renumbering them).

Separately, found and fixed an unrelated pre-existing bug while cross-checking cited keys against the `bib` file: `@Jahagirdar:2017aa` in `writing/cihr-ror-proposal.qmd` used a stray BibDesk/JabRef-style key (with a `:2017aa` suffix baked into the citation itself) instead of the file's

Zotero-derived authoryearN convention, so it never actually resolved even before this session’s regression. Renamed the bib entry to jahagirdar2017 and fixed the citation to match.

Verified with a full audit: extracted every @key cited across the three documents that declare bibliography: ror.bib (cihr-ror-proposal.qmd, proposal.qmd, ror-modeling-strategy.qmd) and confirmed each resolves exactly against a bib entry (the only non-matches were @fig-/@tbl- cross-reference labels, not citations). Rendered all three to their relevant formats (cihr-ror-proposal.qmd and proposal.qmd to html; ror-modeling-strategy.qmd to html and typst/PDF) – zero citation warnings, zero (?) broken-citation markers in any output.

Appendix added to writing/ror-modeling-strategy.qmd: full ror-sim-deviate.R script

Picked up the “next thing to do” flagged above. Added a new # Appendix: Aim 1 simulation script (ror-sim-deviate.R) section right before # References, implementing the plan already agreed: pull the actual script content in via a knitr file chunk option rather than retyping/excerpting it, so the appendix always reflects whatever the script currently says.

Mechanism: a single chunk,

```
#| label: appendix-sim-deviate
#| file: !expr here::here("code", "ror-sim-deviate.R")
#| eval: false
#| echo: true
#| code-summary: "code/ror-sim-deviate.R -- full script"
```

file (a knitr chunk option, not a Quarto-specific one) reads the named file’s contents in as the chunk’s code at render time; !expr is needed because the value has to be an evaluated R expression (here::here(...)), not a literal string – a bare string path would have looked in the wrong working directory once rendered. eval: false keeps the script from actually executing during render (it has side effects – writes data/sim-deviate.csv – and depends on packages/state we don’t want tied to this document’s render). echo: true is there because the docx format block in this document’s YAML sets execute: echo: false as a document-wide default – without the per-chunk override the appendix would render as an empty code block in Word specifically, even though html/typst were unaffected.

Prose: rather than interleaving explanatory text between fragments of the script (which would require either hand-splitting the single file= inclusion – defeating the point – or adding read_chunk() markers to the script itself just for this), added one framing block before the code that walks through the script’s own seven numbered ## N ... sections at a summary level (parameters and their real vs. placeholder status, candidate-pool/reviewer-scoring generation, the streamlining rule and the dropped “bring back” stage, the two-part deviation model, the internal stopifnot() guardrails, the empirical diagnostic printouts, and the output). Full section-by-section rationale for why the script does what it does was already covered earlier in this log and in the script’s own inline comments – the appendix intro doesn’t repeat that, just orients a reader before they hit ~380 lines of code.

Verified: rendered to html, typst/PDF, and docx. Confirmed via the knitted output (not just visual inspection) that the full script – including its final `write_csv()` call – appears exactly once in each format’s real document body (not just in the html version’s separate “View Source” modal from code-tools, which dumps the raw .qmd and is unrelated), correctly R-syntax-highlighted and foldable in html.

2026-08-21

Expertise reference category switched from medium to high

For simplicity, `exp`’s reference level is now high (was med) in both `ror-sim-deviate.R` and `ror-analysis-score-models.R`. Not just a relabeling – `a0/a2/a3/a4` were re-derived so every category’s *true* probability is unchanged, just re-expressed relative to a different baseline: `a0` = -0.5 (was -0.8), `a2` = -0.3 (med vs. high, was +0.3 high vs. med), `a3` = -0.5 (low vs. high), `a4` = -0.8 (none vs. high). Verified algebraically (recomputing each category’s implied logit from both parameterizations gives identical values) and confirmed downstream: `d1`’s factor level order updated to `c("high", "med", "low", "none")`, `term_labels/truth` in the analysis script updated to match (`expmed/explow/exptime`, all “vs. High Expertise”).

Separately, revisited (again) whether higher expertise means *more* or *less* likely to deviate from consensus – discussed two competing theories: expertise-as-anchoring (confident initial read, doesn’t move much – the original assumption) vs. expertise-as-standing-to-disagree (no expertise = no grounds to argue for a different number = defers to consensus by default). No way to adjudicate from first principles; Sam’s own reviewing experience favours the second story, so `a2-a4` were flipped to make higher expertise mean *more* deviation, not less (`a2=0.3`, `a3=-0.2`, `a4=-0.5` before the reference-category re-derivation above folded these into the new baseline).

Reviewer assignment now correlated with self-rated expertise

Previously `job` (reviewer vs. panelist) and `exp` were assigned independently – unrealistic, since Scientific Officers assign the 3 reviewers based on relevant expertise, not at random. Checked against real reviewer-assignment/expertise data (kept out of this log per standing practice for real committee data): $P(\text{assigned as reviewer} \mid \text{expertise})$ was heavily concentrated in high/medium and nearly zero for low/none. Rather than use those exact figures (one meeting, real cycle-to-cycle variation almost certainly exists), rounded to illustrative weights preserving the same pattern: `exp_reviewer_weight = c(high=40, med=30, low=5, none=1)`, used as `sample()` selection weights for a weighted-without-replacement draw of the 3 reviewers per application (replacing the old fixed-count permutation, which only worked when every application had exactly the same category counts). `exp` itself also switched from a fixed-count permutation to a weighted sample *with* replacement, since group size (`n()`) is no longer constant once `mem_n` varies (see below) – realized category counts now vary stochastically around the intended proportions per application-committee, rather than landing on them exactly, which is arguably more realistic anyway.

Committee member count (`mem_n`) now varies too, and is derived from `pool_size`

Found CIHR's live Fall 2025 (202509PJT) Project Grant committee roster page (cihr-irsc.gc.ca/e/54732.html) and parsed it directly (Chairperson / Scientific Officers / Reviewer-in-Training participants / Members are each separately listed per committee) – 66 committees, “Members” count ranging 8-37 (mean 23.8, median 24, SD 6.5). Striking independent validation: `mem_n` = 24, the project's original placeholder guess, turns out to be almost exactly the real mean/median – but just as wrong to treat as fixed as `pool_size` was before 2026-08-14.

Initially gave `mem_n` its own independent beta draw matching this real distribution (`mem_min`=8, `mem_max`=37, shape chosen to match mean 23.8 / SD 6.5). Sam pushed back: `pool_size` and `mem_n` shouldn't be independent – in practice CIHR sees real application volume by the Fall deadline, then recruits committee members to keep per-member workload manageable, so committee size is *caused by* application volume, not just correlated with it by coincidence. That's a better argument than “add a rejection-sampling guard against implausible combinations” (which was tried first, as a middle ground, and works but only trims the extreme tail rather than reflecting the actual causal story).

Settled on: `mem_n` derived from `pool_size` via a workload target (`target_reviews_per_member`), with lognormal noise so it isn't a deterministic ratio: `mem_n` = `round(3 * pool_size / target_reviews_per_member * exp(rnorm(n, 0, mem_noise_sd)))`, redrawn (not clamped) if outside [`mem_min`, `mem_max`]. `target_reviews_per_member` = 5.5 – between Sam's general recollection (“5-6” applications reviewed per member per competition) and a specific real CIHR invitation email quoted verbatim (“approximately eight to ten applications per competition”), which turned out to be somewhat higher than what combining the independently-calibrated `pool_size` and real `mem_n` distributions alone implied (~5.4 median) – real, unresolved tension between these two real data points, not fully reconciled, 5.5 is a reasonable middle value given both. `mem_noise_sd` tuned from an initial 0.25 down to 0.15 after visual inspection (Sam: “looks a little more sensible to me”) – final correlation between `pool_size` and `mem_n` is $r \sim 0.7$ -0.75.

Added a diagnostic figure to `writing/ror-modeling-strategy.qmd` (`fig-cmte`: total applications, discussed applications, and member count per committee, sorted by member count) built from real aggregated data/`cmte_pool`, not illustrative numbers. `cmte_pool` (`pool_size`/`mem_n` before streamlining) is now saved to `output/cmte-pool.rds` by `ror-sim-deviate.R` and read directly by the `qmd`, rather than hand-recreating the generating code there – avoids yet another hand-synced duplicate of parameters that would drift, the same failure class hit repeatedly this project (see 2026-08-19 `aid/cid` entries, `sim-data-aim1.csv` vs `sim-deviate.csv`, etc.).

Empirical check: fraction of applications discussed per committee ranges 24.1%-41.0% across the 50 committees (median 37.8%) – clustered near 40% for most committees (consistent with the streamlining rule's 60th-percentile-rank structure), with a longer left tail for smaller pools where one discrete application makes a bigger percentage difference.

Model-specification equation added, and a three-format rendering saga

Wrote out `m1_deviate`'s full model specification (likelihood, linear predictor, priors) in the McElreath/u2-sibs-style aligned-array format, for writing/`ror-modeling-strategy.qmd`'s "Model specification (Aim 1)" section. Caught two real errors in an initial draft before it went in: (1) reusing d_{ijk} for the binary "did they deviate" indicator collides with the document's own established meaning of lowercase d_{ijk} (the continuous score differential) – switched to uppercase D_{ijk} for the indicator; (2) a single $\alpha_j \sim \text{Normal}(\text{bar_alpha}, \text{sigma_alpha})$ prior line can't cover three separate grouping factors (`cmte/cid/aid`) – following McElreath's own multi-cluster convention (chimpanzees model, section 13.3.1, shown by Sam), only *one* cluster type absorbs the population intercept `bar_alpha` into its own prior mean; the other two are mean-zero with their own independent SD. Picked member as the cluster carrying `bar_alpha` (closest analogue to McElreath's "actor" – the repeated-measures unit), though the choice is mathematically arbitrary and any of the three would work.

Getting this to render consistently across `html/typst/docx` took real work, and surfaced a durable finding about this document's toolchain (same underlying issue as the 2026-07-15 `vphantom` saga, now characterized precisely):

- **html:** MathJax renders `\begin{array}{rcl}\dots\end{array}` natively, no issue – but at full size it visually bled into the left-hand TOC once `toc-location: left` was set. Fixed by wrapping the equation in `<div style="font-size: 0.8em;">`; verified via direct DOM measurement (not a screenshot – the browser tool's screenshot mechanism had an unrelated rendering bug on this specific MathJax content, returning blank images despite the DOM being correctly populated) that the equation's bounding box now sits entirely within the content column (x: 301-937.5) without overlapping the TOC (right: 273).
- **typst:** pandoc *does* auto-translate array to native Typst math (not a straight failure), but the column spacing comes out wrong – a large, uneven gap after the first column instead of tight alignment. Fixed with a hand-written Typst-native block (`{=typst}` raw block, guarded by `:: {.content-visible when-format="typst"}`), using the same `&/&&&` column-skipping idiom already established in this doc's earlier u2-sibs-derived underbrace block, adapted from `probit(qnorm)` to this project's actual logit link where relevant elsewhere. Verified by compiling to PDF and rendering to PNG directly (`fitz/PyMuPDF`) since no PDF-viewing tool was available – confirmed tight, correct alignment, matches the working isolated test exactly once embedded in the real document.
- **docx:** confirmed *precisely* what breaks, rather than guessing – tested array with/without the description column, and aligned instead of array; all three produce the exact same failure (pandoc's docx/OMML converter cannot convert *any* multi-row `&/\`-aligned math construct, regardless of column count or content – it silently dumps the raw LaTeX source as literal text instead). Only genuinely separate, non-aligned equations survive. Given that, embedded a rendered PNG instead (`writing/media/eq-m1-deviate.png`, generated by compiling the same Typst source standalone with `#set page(width: auto, height: auto)` and rasterizing at 300dpi) rather than degrading to a no-alignment/no-description text fallback – verified via the docx's own XML (image byte-identical to the source PNG, correctly placed immediately after

the intro sentence, correctly proportioned at 5.83in x 2.20in). **Not yet fixed:** the older $E[d_{ijk}] = \underbrace{P(\text{deviate})} \times \underbrace{E[\cdot]}$ equation in “Why a two-part model” has the same docx problem and never got a format-specific fallback – flagged, not fixed.

In the course of this, also fixed two unrelated live bugs caught along the way: a data vs. d variable-name mismatch in fig-cmte (pasted from a scratch script using a different variable name, broke the whole render with an opaque “no applicable method for ‘group_by’ applied to an object of class ‘function’” error, since unassigned data fell through to base R’s data() function), and an orphaned } in ror-analysis-score-models.R (leftover from removing an if (FIT_MODELS) { wrapper around m1_magnitude, left the whole script unparseable via a clean Rscript/source(), though usable if run line-by-line interactively, which is presumably how it went unnoticed).

Magnitude model priors: tested an informed-threshold alternative, not adopting it

Sam fit m1_magnitude (the ordinal cumulative() part) and found it slow. Considered following the threshold-prior approach from a Solomon Kurz blog post (solomonkurz.netlify.app/blog/2023-05-21-causal-inference-with-ordinal-regression/) – individual Normal priors per threshold (via coef = 1..9 within class = Intercept) instead of one flat/generic prior across all of them, rationale being that default/diffuse threshold priors are poorly identified by sparse extreme categories (our magnitude outcome is heavily concentrated near +/-0.1, sparse at +/-0.5), which is a plausible driver of slow, inefficient sampling.

Adapted (not copied) the blog’s method: their example uses a probit link and assumes *equal* category probability as a neutral default; ours uses cumulative(link="logit") and we have much better information than “assume uniform,” so thresholds were centered via qlogis(cumsum(p)) on the real simulated category proportions instead.

Tested empirically rather than just adopting it – fit m1_magnitude both ways (generic single prior vs. the 9 informed threshold priors), same data/formula/seed, then compared:

- Divergences / max treedepth: 0/0 in both – already clean.
- Fixed effects (job/exp): essentially identical between the two.
- Bulk ESS: meaningfully better with informed priors (minimum across fixed effects 2267 -> 3645, ~61% improvement) – the theoretical rationale held.
- Wall-clock time: generic 773.7s, informed 970.2s – ~25% *slower* with the informed priors, the opposite of the hoped-for result. Better per-draw mixing efficiency didn’t translate into less total compute, apparently because the different posterior geometry costs more per iteration. Single run each, so some of this could be run-to-run noise, but it’s certainly not the clear speed win the theory predicted.
- LOO: elpd_diff = -1.2, se_diff = 1.1 (loo_compare()), all Pareto-k < 0.7 for both – statistically indistinguishable predictive fit either way.

Decision: not adopting the informed threshold priors – they don’t hurt (identical inference, reliable LOO, zero pathological sampling issues in either version), but they don’t solve the actual problem (wall-clock speed) either, so there’s no reason to add the complexity right now. Flagged

`adapt_delta` (currently 0.95, conservative, with zero divergences observed at that setting in either model – real headroom to lower it) as a more promising lever for speed specifically, since it directly controls average leapfrog steps per iteration rather than threshold identifiability, but not tested yet – explicitly deferred, not pursued today.

2026-08-25

Job-specific magnitude SD added to `ror-sim-deviate.R`: first treatment effect on the magnitude model

Until now `job` (reviewer vs. panelist) only entered Part 1 of the two-part model (`p_dev`, via `a1`) – panelists deviate more often, but conditional on deviating, magnitude was drawn from the same `dev_sd = 0.15` for everyone. Wanted to start experimenting with a treatment effect on magnitude too: given that panelists already deviate more often, do they also swing harder when they do?

Working from real `m1_magnitude` marginal predictions (`job` showed essentially no separation at any category), the ask was for a plausible, non-extreme, detectable difference – illustrated hypothetically as “reviewer ~15%, panelist ~18%” probability of landing in the 0.2 category specifically, conditional on deviating.

That specific target turns out not to be reachable by scaling SD alone. The single-category probability of landing exactly at 0.2 is *not* monotonic in `sd` – for a truncated-normal-then-rounded magnitude on $[-0.5, 0.5]$, it peaks around 15.3% near `sd=0.20` and then *declines* as `sd` grows further, because additional spread keeps pushing mass further out to 0.3/0.4/0.5 instead of piling more onto 0.2. No value of a job-specific `dev_sd` can hit “18% at exactly the 0.2 bin” while reviewers sit at 15%.

Resolution: reframed the target as tail mass instead of a single category. Introduced `dev_sd_reviewer = 0.15` (unchanged) and `dev_sd_panelist = 0.30`, applied via a per-row `dev_sd_i` vector in the existing `rtruncnorm()` call (both the main draw and the redraw-if-lands-on-zero loop). $P(|\text{deviation}| \geq 0.3)$ is the natural summary statistic here since (unlike the single 0.2 category) it increases monotonically with `sd`.

Realized in the regenerated `data/sim-deviate.csv`, conditional on `deviated == 1`:

<code> deviation </code>	reviewer	panelist
0.1	50.1%	32.0%
0.2	30.7%	27.4%
0.3	12.5%	21.9%
0.4	5.7%	14.4%
0.5	1.0%	4.4%
$P(d \geq 0.3)$	19.2%	40.6%
<code>mean(d)</code>	0.177	0.232

Note this realized contrast came out larger than a naive hand calculation predicted, because that calculation used $\text{mean} = 0$ and missed that the DGP actually centers each draw on $\text{dev_bias} + u0m_bias$ (the existing member-level leniency/harshness term, $\text{sd} = 0.1$) – between-member heterogeneity in the mean adds real extra population-level spread beyond dev_sd alone. Caught by Monte Carlo- checking the exact hierarchical structure against the simulated output before reporting the numbers as final.

Not yet done: refitting `m1_magnitude` on the regenerated data to confirm the two-part model actually recovers this job effect on magnitude, rather than assuming it will.

Aside: `MASS::select()` masking `dplyr::select()` broke the sim script mid-session

`HitError in select(cmte_pool, -mem_n_center) : unused argument (-mem_n_center)` when re-running `ror-sim-deviate.R` in a session that had `MASS` loaded earlier (for ordinal/polr work). `MASS` exports a real, non-generic `select()` (for `MASS::select.polr`), which – unlike a normal S3 method – fully shadows `dplyr::select` if loaded afterward, rather than dispatching through it. Script itself was fine (confirmed clean in a fresh Rscript session); fixed by detaching `MASS`. Worth remembering next time any sim script throws an “unused argument” error on a `select()` call that looks syntactically correct – check `find("select")` before assuming the script broke.

`m1_magnitude` couldn’t see the job effect on magnitude: needed a `disc` (dispersion) submodel, not a location fixed effect

Followed up on the “not yet done” item above. Refit `m1_magnitude` on the regenerated data/`sim-deviate.csv` (the one with `dev_sd_reviewer/dev_sd_panelist`), and the `jobpanelist` coefficient came back at ~ 0 (estimate 0.015, CI comfortably spanning zero) despite the underlying data showing a large, real split: $P(|d| \geq 0.3) = 19.2\%$ (reviewer) vs. 40.6% (panelist), exactly matching the numbers above. Confirmed this wasn’t a stale-fit or stale-data problem first – loaded the fitted model’s own stored data (`m1_magnitude$data`) and verified it reproduced those exact percentages, so the model really was fit on the current simulation output.

Diagnosis: `dev_sd_reviewer/dev_sd_panelist` is a pure *dispersion* effect – same center, different spread. Confirmed directly: `mean(signed deviation)` is ~ 0 for both roles (-0.005 reviewer, -0.004 panelist) while SD differs substantially (0.201 vs. 0.260). A standard `cumulative(link = "logit", threshold = "flexible")` model is proportional-odds: every fixed effect enters as a single linear predictor that shifts the *location* of the latent distribution uniformly across all thresholds. It has no way to represent “same center, wider tails” – a symmetric widening doesn’t correspond to any net directional shift, so the location-only coefficient on `job` correctly converged to ~ 0 . The model wasn’t wrong; it just couldn’t see the parameter that was actually varied.

Fix: added a `disc` (discrimination) submodel – `cumulative()`’s auxiliary dispersion parameter, a multiplicative scale on the linear predictor and thresholds together, $\text{disc} \sim 1/\text{SD_latent}$ in effect. Modeling $\text{disc} \sim 0 + \text{job}$ lets reviewer and panelist have their own scale, which is the parameter the simulation actually manipulates:

```
bf(deviation ~ 1 + job + exp + (1 | cmte) + (1 | cid) + (1 | aid),
   disc ~ 0 + job)
```

with an added prior on the new parameter block (`prior(normal(0, 1), class = b, dpar = disc)`, since `disc`'s coefficients need a prior specified separately from the main `class = b` block). Fit as `m2_magnitude`, cached to `code/fits/ror-magnitude-m2.rds` (kept `ror-magnitude-m1.rds`, the location-only fit, as-is rather than overwriting it, since it's still the right model for `exp`, which doesn't have a simulated dispersion effect).

Hit one real mistake along the way, not just a modeling gap: the first attempt at `m2_magnitude` threw Argument 'autocor' should be specified within the 'formula' argument and `disc` came back fixed at exactly 1.00 with `Rhat/ESS = NA` (not a convergence issue – `NA` there means `disc` was never sampled at all, just held at its default constant). Cause: `disc ~ 0 + job` had been passed to `brm()` as a separate argument rather than bundled into the same `bf()` object as the main formula, so it landed in `brm()`'s unrelated (and largely legacy) `autocor` slot and was silently dropped as a model term. Fixed by moving both formulas into one `bf(...)` call.

Result, refit correctly: `disc` came out clearly separated by role (reviewer more discriminating/concentrated, panelist flatter), and `avg_predictions(m2_magnitude, variables = "job", re_formula = NULL, ndraws = 200)` now shows the expected shape: reviewers concentrated near ± 0.1 , panelists visibly heavier in the ± 0.3 - ± 0.5 tails. Visualized as a dodged bar chart (predicted probability by signed deviation category, colored by role) rather than a line/ribbon, since the x-axis is 10 discrete categories with a real gap at zero, not a continuous dose axis – bars read as two overlaid probability mass functions, which is what these are. Sam is adding this figure to `writing/ror-modeling-strategy.qmd` directly.

Cached the magnitude-model marginal predictions to output/, matching m1-deviate-me.rds's pattern

`ror-analysis-score-models.R` already saved `m1_deviat`'s `avg_predictions()` output to `output/m1-deviate-me.rds` so `ror-modeling-strategy.qmd` can read a small cached object instead of loading `brms/cmdstanr` and the full fit just to build a table. Did the same for the two magnitude-model predictions object from this session:

- `output/m1-magnitude-exp-me.rds` – `exp`'s predicted magnitude distribution, from `m1_magnitude` (the location-only fit). `exp` has no simulated dispersion effect, so the original model is still the right one for it; only `job` needed the `disc` submodel.
- `output/m2-magnitude-job-me.rds` – `job`'s predicted magnitude distribution, from `m2_magnitude` (the `disc`-aware fit), matching the dodged-bar figure above.

Both follow the same “rebuild a bare tibble from `$`-extracted columns” trick `m1-deviate-me.rds` already used: a `marginalEffects` predictions object carries a hidden ~140MB attribute (the full model/draws context) that survives `select()/mutate()/as_tibble()`, so `saveRDS()` on what looks like a small table silently balloons the file unless you rebuild a fresh tibble from the raw column vectors first (which drops unrecognized attributes). Verified both save at ~1KB, not ~140MB, and read back correctly.

2026-08-26

ror-analysis-score-models.R split into a deviate script and a magnitude script

The combined script was getting long (priors, m1_deviate, its table and marginal effects, then both magnitude models and theirs, all in one file). Split along the same Part 1 / Part 2 line the modeling strategy doc already uses:

- code/ror-analysis-deviate-model.R – priors, m1_deviate (bernoulli), its table, its marginal effects.
- code/ror-analysis-magnitude-model.R – the magnitude models and their marginal effects.

Package-load/data-read boilerplate is duplicated across both rather than shared, matching this project’s existing convention of self-contained per-piece scripts (see “A note on simulation architecture”). Output paths (output/m1-deviate-*.rds, output/m1-magnitude-exp-me.rds, output/m2-magnitude-job-me.rds, code/fits/*) are unchanged. Verified both new files parse cleanly via parse() rather than actually running them – the “delete cached fit” line at the top of each model section forces a full multi-minute refit on every run, which wasn’t worth triggering just to check a mechanical split.

Two real, pre-existing render-breaking bugs surfaced while updating writing/ror-modeling-strategy.qmd’s references to the old filename (unrelated to the split itself, just uncovered by re-rendering):

1. A duplicate chunk label (tbl-m1-deviate reused for both the Part 1 table and the Part 2 plot) – Quarto errors on this rather than silently overwriting. Renamed the Part 2 one to fig-m2-magnitude-job (it’s a plot, not a table – looked like a copy-paste leftover).
2. The Part 2 plot chunk called avg_predictions(m2_magnitude, ...) live, but m2_magnitude only ever exists inside an eval: false display chunk – never actually in the render session. Fixed to read the already-saved output/m2-magnitude-job-me.rds instead, matching the pattern used everywhere else in this doc.

Both fixed; html and docx render clean end-to-end.

Flagged, not fixed: ror-analysis-magnitude-model.R has a duplicate model block and a tibble-recycling bug

While reviewing the script for the split above, found it in a mid-refactor state (Sam’s own direct edit, not touched by me): a duplicate m1_magnitude-fitting block (the new disc ~ 0 + job model at the top, an old location-only block still below it, both targeting code/fits/ror-magnitude-m1.rds – this inverts the established naming from 2026-08-25 above, where the disc-aware fit was m2_magnitude and m1_magnitude was location-only), and a real bug in the d_magnitude_job construction: it discards the just-computed avg_slopes() result and pulls estimate/conf.low/conf.high from p_magnitude_job (the predictions object, 20 rows) while keeping group from the slopes object (10 rows) – confirmed directly that tibble() errors on this length mismatch

(Tibble columns must have compatible sizes), so a clean run would halt right there and never reach `saveRDS()`.

The currently-saved output/m2-magnitude-job-me.rds (which the two figures already in the modeling-strategy doc read from) looks correct – genuine `avg_slopes()`-shaped contrasts, sensible signs – so it must be left over from an earlier working version of the script; re-running the script as it stands now would not reproduce it. Sam asked to leave the script alone for now and just do the doc write-up; this is flagged here so it isn’t mistaken for current/working when someone next opens that file.

Added the magnitude model’s equation (html/typst) and a justification for ordinal over categorical

Added a “Model specification: magnitude, given a deviation” section to `writing/ror-modeling-strategy.qmd`, right after Part 1’s equation and prior plot, in the same style (html array block, native typst raw block, no docx variant since that wasn’t asked for – docx will just skip the section rather than error).

One real difference from Part 1’s equation, not just a relabeling: a cumulative ordinal model’s flexible thresholds already anchor location, so none of the three random intercepts (`cmte/cid/aid`) needs to absorb a population mean the way `alpha_mem` did in Part 1 – all three are mean-zero here. The other addition is the `disc ~ 0 + job` role-specific discrimination term from 2026-08-25 above, written as `log(kappa_ijk) = delta^disc_Reviewer * Reviewer + delta^disc_Panelist * Panelist` (both levels get their own coefficient directly, matching `0 + job`’s no-intercept coding).

Also added prose justifying `cumulative()` over an unordered categorical/multinomial model for the magnitude outcome, prompted by Sam asking directly: is “order” here just an artifact of how we labeled things, or real? Four points, in order of how much they actually matter:

1. The 10 levels are the literal numeric ordering of `final_score` – consensus, not arbitrary labels – categorical throws that away.
2. Matches the true mechanism: a continuous truncated-normal draw rounded to one decimal place, i.e. a discretized continuum, which is the standard case for an ordinal (not categorical) model.
3. Parameter efficiency: categorical needs a full separate coefficient set per non-reference category (~9x the parameters), most poorly identified given how sparse the tail categories are; ordinal shares one location (and now one dispersion) parameter across all 10 via the threshold structure.
4. The actual hypotheses (“panelists deviate larger on average,” “panelists show more spread”) are inherently location/dispersion statements, which only have a natural expression on an ordered scale – categorical has no such notion at all.

Flagged one honest caveat: treating the *full signed* scale as one ordered dimension (rather than direction as a separate unordered bernoulli, magnitude as ordinal `|d|` – the three-part-hurdle alternative discussed on 2026-08-24) is an assumption, not something forced by the data. Chose

it because it matches how these scores actually arise (one continuous departure from consensus, not “decide direction, then decide how far”), not because it’s the only defensible option.

Caught and fixed one small issue along the way: an inline $|\text{rm magnitude}|$ didn’t convert cleanly to typst (rm isn’t supported; pandoc fell back to raw TeX with a warning) – switched to $\text{text{magnitude}}$, matching how the rest of the doc writes text inside math. Verified the fix by compiling the generated .typ directly against the repo root (`typst compile --root .`), which bypasses the still-unresolved, pre-existing sandbox restriction on `output/ror-priors-m1-deviate.png` escaping writing/’s directory (flagged 2026-08-25/26, not yet fixed) – confirms the document content itself compiles clean; the image-path issue is a separate, already-flagged problem.

2026-08-27

Pinned down the exact mechanism of the `marginaleffects` hidden-attribute bug – earlier documentation of it was over-broad

Came back to this after noticing `output/m2-magnitude-job-me.rds` had ballooned to 126MB again in a commit (see the push-failure fix, same day). Wanted to know precisely whether `as.data.frame()` vs. `as_tibble()` mattered, and whether the manual bare-vector `tibble()` `rebuild(tibble(group = x$group, ...))` used in both analysis scripts was actually the *only* way to fix this, since it reads as distinctly non-tidy code.

Tested `as_tibble()` as a drop-in replacement for `as.data.frame()` first: doesn’t help. Loaded the cached `m1_magnitude` fit and compared `as.data.frame(raw)`, `as_tibble(raw)`, and the manual rebuild directly: both `as.data.frame()` and `as_tibble()` preserve the hidden attribute (133.6MB, identical to the untouched object); only the manual bare-vector rebuild strips it (3.6KB). Makes sense once you know why: `avg_predictions()` output already inherits `data.frame(class = c("predictions", "data.frame"))`, so both coercion functions are close to a no-op reclassification on an already-`data.frame`-like object, not a rebuild – neither has any reason to know this particular attribute should be dropped.

But then tested `select()/mutate()` called directly on the raw object (no `as.data.frame()` first), expecting them to also fail based on this project’s own prior documentation (“survives `select()/mutate()/as_tibble()`”) – and they didn’t fail. Both stripped the attribute cleanly, on both `avg_predictions()` and `avg_slopes()` output, alone or combined via `bind_rows()` first. That directly contradicts what’s written in `reference_ror_docs.md` and in comments in both analysis scripts, so this needed tracing properly rather than just updating the docs to match the new observation – if the earlier claim was wrong, something about *how* it was wrong needed to be understood, not just the correct answer.

Root cause, isolated by testing the exact original bug chain step by step: the critical factor is whether `as.data.frame()` was ever called *before* `select()/mutate()`, not whether they’re called at all.

Chain	Attribute survives?
<code>avg_predictions() \> select()</code>	No
<code>avg_predictions() \> mutate()</code>	No
<code>avg_predictions() \> as.data.frame() \> select()</code>	Yes
<code>avg_predictions() \> as.data.frame() \> mutate()</code>	Yes
<code>avg_slopes() \> select()</code>	No

Once `as.data.frame()` demotes the object to a bare `data.frame` (losing the `predictions/slopes` class), every subsequent `select()/mutate()/bind_rows()` call preserves the attribute from then on – a bare `data.frame`’s dplyr verbs don’t rebuild/strip attributes the way the special `marginalEffects` classes’ do. The original bug’s exact code (`avg_predictions(...) |> as.data.frame() |> mutate(...)`, in both `p_overall/p_exp/p_job` and the `m2-magnitude-job-me.rds` construction) called `as.data.frame()` first in every case, which is exactly why the `select()/mutate()` steps later in the same pipeline didn’t help – by then it was already poisoned. The `as.data.frame()` calls were never doing anything useful in the first place; `mutate()/select()` work fine directly on `marginalEffects` output.

Fix applied to `ror-analysis-deviate-model.R`: removed the three `as.data.frame()` calls (on `p_overall/p_exp/p_job`), which makes the pre-existing manual bare-vector `tibble()` rebuild step entirely redundant – deleted it. The existing `bind_rows() |> select() |> mutate()` pipeline that was already there now does the whole job on its own. Verified against the real cached `m1_deviate` fit before and after: identical 7-row content, `saveRDS()` size 376 bytes either way, hidden attribute confirmed absent. `ror-analysis-magnitude-model.R` has the same `as.data.frame()` pattern but was left alone, consistent with the 2026-08-26 decision not to touch that file until its other issues (duplicate model block, `d_magnitude_job` bug) get sorted out in one pass.

Updated `reference_ror_docs.md` to replace the old, over-broad characterization of this bug with the precise mechanism above.

Why the magnitude model’s equation shows two disc coefficients for a binary covariate

Sam asked a sharp question about the modeling-strategy doc’s role-specific-discrimination line: role (job) is binary, so why does $\log(\kappa_{ijk})$ show *two* parameters ($\delta_{Reviewer}^{disc}$, $\delta_{Panelist}^{disc}$) when the location line (η_{ijk}) only needed one (δ , for Panelist, with Reviewer implicit at 0)?

Answer: it’s a coding-convention difference, not extra information. A 2-level factor only ever spans 2 free numbers no matter how it’s written. η_{ijk} uses standard $1 + \text{job}$ treatment coding (intercept absorbed elsewhere by the mean-zero random effects/ thresholds structure; Reviewer implicit at 0, Panelist gets one contrast δ). The disc submodel was specified in code as `disc ~ 0 + job` (no intercept) instead, which under R’s model-matrix rules gives one *full* dummy per level rather than intercept + contrast – so both $\delta_{Reviewer}^{disc}$ and $\delta_{Panelist}^{disc}$ appear, each directly equal to that role’s own $\log(\kappa)$.

Why $\theta + \text{job}$ was chosen for disc specifically (this was actually already the rationale in the 2026-08-25 entry above, worth restating plainly): unlike location, dispersion has a natural “no effect” reference that isn’t tied to either factor level – brms fixes $\kappa = 1$ for everyone whenever no disc formula is specified at all. $\theta + \text{job}$ lets both roles’ coefficients be read directly against that same neutral default, rather than one role’s coefficient being an absolute value and the other being a ratio relative to it.

A real subtlety surfaced while answering this, worth recording precisely: a naive $1 + \text{job}$ -with-implicit-zero-Reviewer coding for disc (mirroring η ’s exact convention) would NOT be an equivalent relabeling the way it is for location – it would pin Reviewer’s discrimination at exactly $\kappa = 1$ with no freedom to move, which is a genuinely more restrictive model than what’s fit ($\theta + \text{job}$ estimates *both* roles’ discrimination freely). The reason this works harmlessly for η but not for κ : η has other structure (random effects, thresholds) that implicitly absorbs a reference value with no real constraint being added; κ has none of that, so forcing Reviewer’s coefficient to exactly 0 would be a real constraint, not a notational choice.

Updated the modeling-strategy doc’s disc equation to $1 + \text{job}$ -style notation, for consistency with η

Sam asked for this switch specifically in how the model is *written* in `writing/ror-modeling-strategy.qmd` (not a change to the actual brms formula, which stays $\text{disc} \sim \theta + \text{job}$). Given the subtlety above, implemented it as a genuine intercept-plus-contrast form rather than mirroring η ’s implicit-zero convention: $\log(\kappa_{ijk}) = \delta_0^{\text{disc}} + \delta_1^{\text{disc}} \text{Panelist}_{ijk}$, where both δ_0^{disc} (Reviewer’s own log-discrimination) and δ_1^{disc} (Panelist’s contrast from it) remain freely estimated – same 2-parameter space as $\theta + \text{job}$, just relabeled, not the more restrictive implicit-zero version. Added explicit prose in the doc flagging that this is a presentational choice for consistency with η_{ijk} ’s style, distinct from the actual fitted formula, with a pointer to this log entry for the full reasoning. Verified both html and typst render clean after the change (typst checked via `direct typst compile --root .`, same pre-existing image-path issue as before, unrelated to this edit).

Actually fixed the typst image-path issue, instead of just working around it again

This has been flagged as an unresolved, pre-existing blocker several times over the last couple of days (worked around each time via `typst compile --root .` for my own verification, never fixed). Sam hit it directly running `quarto render ... --to typst` himself, which doesn’t have that workaround available, so it was time to actually fix it.

Root cause: `knitr::include_graphics(here("output", "ror-priors-m1-deviate.png"))` emits a relative path (`./output/ror-priors-m1-deviate.png`) in the generated `.typ` file, since `output/` sits one level above `writing/`. Typst’s sandboxed compiler restricts file access to its “project root,” which defaults to the input file’s own directory (`writing/`) when nothing else is configured – so any `../`-escaping path is refused with “access denied ... cannot read file outside of project root.” `quarto render` doesn’t pass a `--root` flag pointing at the repo root, so this bites every plain `quarto render --to typst` call, not just an edge case.

Fix: moved `ror-priors-m1-deviate.png` into `writing/media/` (alongside `eq-m1-deviate.png`, which already lives there for exactly this same reason) instead of `output/`, and updated both the `ggsave()` call in `ror-analysis-deviate-model.R` and the `knitr::include_graphics()` call in `writing/ror-modeling-strategy.qmd` to match. This PNG is only ever consumed by this one qmd (unlike genuine cross-script output/ artifacts like `cmte-pool.rds`), so moving its generation target is clean, not a duplication-drift risk.

Verified for real this time – `quarto render writing/ror-modeling-strategy.qmd --to typst` (no `--root` workaround) now completes with `DONE` and produces a PDF. `html` and `docx` still render clean too.